

ThInf_zsmf

Disclaimer

Zur Erstellung dieses Dokuments wurde sehr viel mit LLMs gearbeitet. Die Informationen können fehlerhaft und unvollständig sein, deshalb unbedingt schon mit einer Grundahnung an die Sache herangehen und nicht ausschließlich die Zusammenfassung zum Lernen, sondern viel eher zum schnellen Wiederholen verwenden!

Generell ist die Auffassung des Stoffs hier sehr verkürzt und zielt ausschließlich auf Wiederholung ab!

Inhalt

1. [Probleme, Berechenbarkeit und Reduktionen](#)
2. [Berechenbarkeit](#)
3. [Formale Sprachen](#)
4. [Komplexitätstheorie](#)
5. [Korrektheit von Programmen](#)

Probleme, Berechenbarkeit und Reduktionen

Teil 1.1: Probleme, Programme, Algorithmen

Probleme: Formale Definition

Definition: Problem

Ein Problem ist definiert durch:

1. Eine (abzählbar) unendliche Menge von möglichen **Instanzen** (Inputs).
 2. Eine **Frage**, die man an diese Instanz stellt.
- **Entscheidungsproblem**: Ein Problem, bei dem die Frage eine **ja/nein** Antwort erwartet.
 - "Ja"-Instanzen = positive Instanzen.
 - "Nein"-Instanzen = negative Instanzen.

Beispiel: GRAPH-ERREICHBARKEIT

- **Instanz**: Ein Graph $G = (V, E)$ und Knoten $u, v \in V$.
- **Frage**: Gibt es im Graph G einen Pfad von u nach v ?

Warum unendlich viele Instanzen?

- Lösungsverfahren müssen allgemein gültig sein (kein bloßes **Look-up** in einer endlichen Tabelle).
- Ermöglicht asymptotische Analyse der Schwierigkeit.

Weitere Problemtypen

Neben Entscheidungsproblemen (ja/nein) gibt es:

- **Funktionsproblem**: Berechne das Ergebnis $f(x)$ einer Funktion f .
- **Optimierungsproblem**: Finde den optimalen Wert (min / max), z.B. Länge des kürzesten Pfades.
- **Suchproblem**: Finde **eine** konkrete Lösung (z.B. einen Pfad).
- **Aufzählproblem**: Liste **alle** Lösungen auf.
- **Zählproblem**: Gib die **Anzahl** der Lösungen an.

Algorithmen & Programme

Definition: Algorithmus

Eine Beschreibung von Rechenschritten, um jede beliebige Instanz eines Problems P zu lösen.

Anforderungen:

1. Muss auf **alle** Instanzen anwendbar sein.
2. Muss die **korrekte** Antwort liefern.
3. Muss nach endlich vielen Schritten **terminieren**.
4. Schritte müssen einfach (maschinell ausführbar) sein.
5. Beschreibung muss (für Menschen) verständlich sein.

Programmiersprache SIMPLE

Als Modell für Berechenbarkeit dient die Sprache "SIMPLE" (Kern imperativer Sprachen: Variablen, Zuweisung, `while`, `if-then-else`).

- **Input:** Liste von Werten oder ein String.
- **Output:** `return` Wert.
- **Wichtig:** Ein Programm terminiert nicht garantiert (partielle Funktion).
- **Angemessenheit:** SIMPLE kann alle Algorithmen ausdrücken, die auch in Java/C möglich sind (Turing-mächtig).

Teil 1.2: Berechenbarkeit & Entscheidbarkeit

Berechenbarkeit (Funktion)

Eine Funktion f heißt **berechenbar**, wenn es einen Algorithmus gibt, der für jeden Input x den Output $f(x)$ liefert.

Entscheidbarkeit (Problem)

Ein Entscheidungsproblem P heißt **entscheidbar**, wenn es einen Algorithmus gibt, der für jede Instanz x korrekt `ja` (`true`) oder `nein` (`false`) liefert und **immer terminiert**.

Unterschied Partielle vs. Totale Funktion:

- Berechenbare Funktionen können partiell sein (d.h. auf manchen Inputs nicht terminieren / nicht definiert sein).
- Für Entscheidbarkeit (totale Funktion) verlangen wir Termination auf **jeder** Instanz.

Existenz unentscheidbarer Probleme

Es gibt **überabzählbar** viele Probleme, aber nur **abzählbar** viele Programme.

→ Es muss Probleme geben, für die kein Programm existiert.

1. Abzählbarkeit von Programmen (Σ^*):

- Alle Programme sind Strings über einem endlichen Alphabet.
- Man kann sie der Länge nach (und lexikographisch) ordnen und nummerieren.
- Es gibt eine Bijektion $g : \Sigma^* \rightarrow \mathbb{N}$.

Aufzählung:

Strings	Anzahl	Positionen
ϵ (leerer String)	1	0
$a_1, \dots, a_k$	k	$1, \dots, k$
$a_1 a_1, a_1 a_2, \dots, a_k a_k$	k^2	$k + 1, \dots, k^2 + k$
$a_1 a_1 a_1, a_1 a_1 a_2, \dots, a_k a_k a_k$	k^3	$k^2 + k + 1, \dots, k^3 + k^2 + k$
etc.		

2. Überabzählbarkeit von Funktionen ($\{0, 1\}^{\mathbb{N}}$):

- Beweis durch **Cantor'sches Diagonalverfahren**.
- Man nimmt an, alle Funktionen seien aufzählbar ($f_0, f_1, \dots$).
- Man konstruiert eine neue Funktion $\hat{f}$, die sich von f_n an der Stelle n unterscheidet:
 $\hat{f}(n) = 1 - f_n(n)$.
- Widerspruch: $\hat{f}$ kommt in der Aufzählung nicht vor.

Aufzählung von $\{0, 1\}^{\mathbb{N}}$:

f_0	=	a ₀₀	a_{01}	a_{02}	a_{03}	a_{04}	...
f_1	=	a_{10}	a ₁₁	a_{12}	a_{13}	a_{14}	...
f_2	=	a_{20}	a_{21}	a ₂₂	a_{23}	a_{24}	...
$\vdots$	=	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	

Das Halteproblem

Ein klassisches Problem über das Verhalten von Programmen. Wäre es lösbar, könnten offene mathematische Fragen (z.B. Goldbachsche Vermutung) durch Brute-Force-Suche entschieden werden.

HALTEPROBLEM

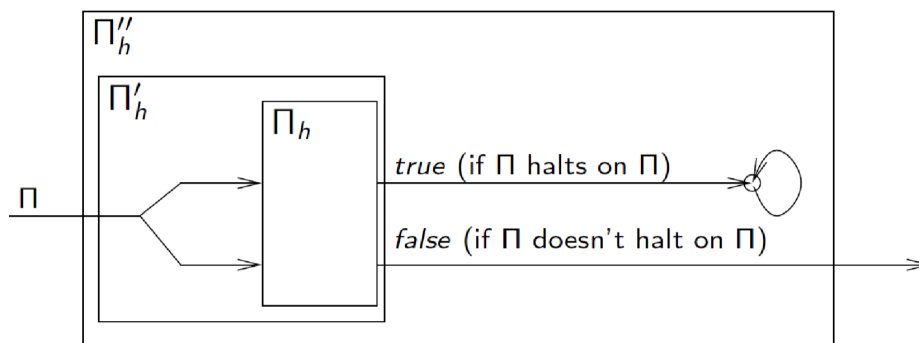
- **Instanz:** Quellcode eines Programms Π und ein Input-String I .
- **Frage:** Terminiert das Programm Π auf dem Input I ?

Theorem

Das HALTEPROBLEM ist **unentscheidbar**.

Beweis (Widerspruch / Diagonalargument):

1. Annahme: Es gibt ein Programm $\Pi_h(\Pi, I)$, das entscheidet, ob Π auf I hält (`true` / `false`).
2. Konstruiere $\Pi'_h(\Pi)$: Ruft $\Pi_h(\Pi, \Pi)$ auf (Prüft, ob Programm auf eigenem Quellcode hält).
3. Konstruiere $\Pi''_h(\Pi)$:
 - Wenn $\Pi_h(\Pi, \Pi)$ `true` liefert $\rightarrow$ gehe in **Endlosschleife**.
 - Wenn $\Pi_h(\Pi, \Pi)$ `false` liefert $\rightarrow$ terminiere (`return false`).
4. Lasse Π''_h auf sich selbst laufen: $\Pi''_h(\Pi''_h)$.
 - Wenn es hält $\rightarrow$ darf es laut Konstruktion nicht halten.
 - Wenn es nicht hält $\rightarrow$ müsste es laut Konstruktion halten.
 - **Widerspruch!**



```
Boolean  $\Pi''_h$  (String  $\Pi$ )
  if  $\Pi'_h(\Pi) = \text{true}$  then { while (true) do {} }
  else return false;
```

Weitere unentscheidbare Probleme

- **KORREKTHEIT**: Terminiert Π auf I_1 und liefert I_2 ? (Impliziert Halteproblem).
- **ERREICHBARER-CODE**: Wird Zeile n jemals ausgeführt? (Impliziert Halteproblem, wenn n das Ende ist).

Teil 1.3: Semi-Entscheidbarkeit

Wir lockern die Bedingung: Was, wenn das Programm nur bei "ja" garantiert antworten muss?

Semi-Entscheidbarkeit

Ein Problem P ist semi-entscheidbar, wenn es ein Programm Π gibt, sodass:

- Ist I eine **positive** Instanz ("ja") $\rightarrow$ Π terminiert und liefert `true`.

- Ist I eine **negative** Instanz ("nein") $\rightarrow$ II liefert `false` **ODER terminiert nicht**.

Beispiele:

1. **HALTEPROBLEM**: Semi-entscheidbar. (Führe Programm einfach aus. Wenn es hält, sag "ja". Wenn nicht, läuft man eh ewig).
2. **KORREKTHEIT**: Semi-entscheidbar (Simulation).
3. **ERREICHBARER-CODE**: Semi-entscheidbar.
4. **ENTSCHEIDUNGSPROBLEM (Prädikatenlogik 1. Stufe)**: Gültigkeit ist semi-entscheidbar (Vollständigkeitssatz von Gödel: Alle gültigen Formeln sind beweisbar $\rightarrow$ Beweise aufzählen).

Einschub: Aufzählung von Paaren

Um "alle möglichen Inputs" oder "alle Zeitschritte" zu prüfen, braucht man eine Strategie für unendliche Mengen ($M_1 \times M_2$).

Geschachtelte Schleifen (`for all inputs { for all steps ... }`) funktionieren nicht, da die innere Schleife evtl. nie endet.

Lösung: Cantor'sches Abzählprinzip (Dovetailing / Zick-Zack durch die Matrix).

Aufzählung von $M_1 \times M_2$ mit $M_1 = \{a_1, a_2, a_3, a_4, \dots\}$ und $M_2 = \{b_1, b_2, b_3, b_4, \dots\}$:

$$\begin{pmatrix} (a_1, b_1) & (a_1, b_2) & (a_1, b_3) & \dots, \\ (a_2, b_1) & (a_2, b_2) & (a_2, b_3) & \dots, \\ (a_3, b_1) & (a_3, b_2) & (a_3, b_3) & \dots, \\ (a_4, b_1) & (a_4, b_2) & (a_4, b_3) & \dots, \\ \vdots & \vdots & \vdots & \end{pmatrix} \quad \begin{pmatrix} 1 & 4 & 9 & 16 & \dots, \\ 2 & 3 & 8 & 15 & \dots, \\ 5 & 6 & 7 & 14 & \dots, \\ 10 & 11 & 12 & 13 & \dots, \\ \vdots & \vdots & \vdots & \vdots & \end{pmatrix}$$

Teil 1.4: Komplement

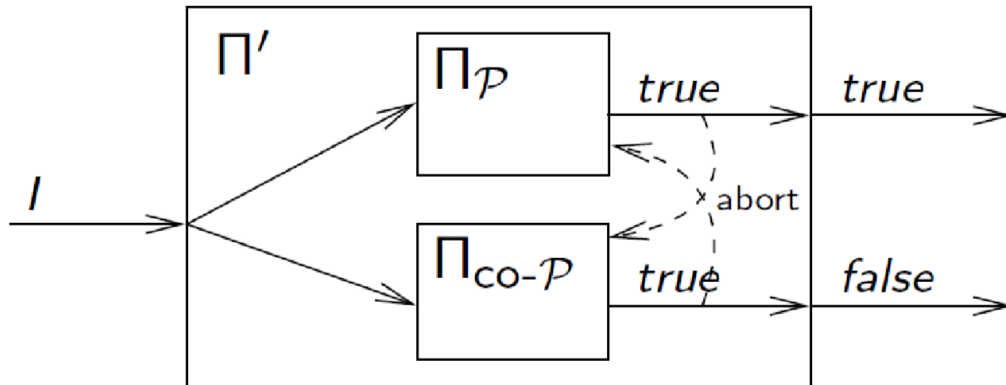
Komplement (co-Problem)

Das co-Problem von P (genannt $\text{co-}P$) hat dieselben Instanzen, aber die **verneinte Frage**.

- **Beispiel:** GRAPH-ERREICHBARKEIT $\leftrightarrow$ NICHT-ERREICHBARKEIT.
- **Eigenschaft 1:** Wenn P entscheidbar $\rightarrow$ $\text{co-}P$ entscheidbar (einfach Output invertieren).
- **Eigenschaft 2:** Wenn P semi-entscheidbar UND $\text{co-}P$ semi-entscheidbar $\rightarrow P$ ist **entscheidbar**.

Beweis zu Eigenschaft 2 (Parallelisierung):

Man lässt den Semi-Entscheider für P und den für $\text{co-}P$ "parallel" (abwechselnd schrittweise) laufen. Einer von beiden *muss* irgendwann `true` liefern (da jede Instanz entweder positiv oder negativ ist). Damit hat man eine sichere Antwort.



Teil 1.5: Jenseits der Semi-Entscheidbarkeit

Gibt es Probleme, die **nicht einmal semi-entscheidbar** sind?

Theorem

Wenn P unentscheidbar, aber semi-entscheidbar ist $\rightarrow$ ist $\text{co-}P$ **nicht** semi-entscheidbar.

- Daraus folgt: **co-HALTEPROBLEM** (Terminiert das Programm *nicht*?) ist nicht semi-entscheidbar.

Es gibt auch Probleme, wo weder P noch $\text{co-}P$ semi-entscheidbar sind (höhere Komplexität, man müsste implizit das co-Halteproblem lösen):

- SELBER-OUTPUT:** Π_1 und Π_2 verhalten sich auf Input I gleich (beide terminieren gleich oder beide nicht).
- PROGRAMM-ÄQUIVALENZ:** Verhalten sich Π_1 und Π_2 auf *allen* Inputs gleich?
- UNIVERSELLES-HALTEPROBLEM:** Hält Π auf *allen* Inputs?

Teil 1.6: Reduktionen

Reduktionen sind das wichtigste Werkzeug, um die Schwierigkeit von Problemen zu vergleichen. Schreibweise: $A \leq B$ ("A ist reduzierbar auf B").

Bedeutung: **B ist mindestens so schwer wie A.** (Kann man B lösen, kann man auch A lösen).

Arten von Reduktionen

- Turing Reduktion:**

- Löse Problem A, indem du ein (hypothetisches) Lösungsverfahren für B als Unterprozedur ("Orakel") aufrufst.
- Beliebig viele Aufrufe erlaubt.

2. Many-One Reduktion (Spezialfall):

- Transformiere Instanz x von A in Instanz $R(x)$ von B.
- x ist Ja-Instanz von A $\iff R(x)$ ist Ja-Instanz von B.
- Das Ergebnis von B wird direkt als Ergebnis für A übernommen (nur 1 Aufruf ganz am Ende).

Ressourcen

Reduktionen müssen in der Berechenbarkeitstheorie *berechenbar* sein (in der Komplexitätstheorie meist *polynomiell*).

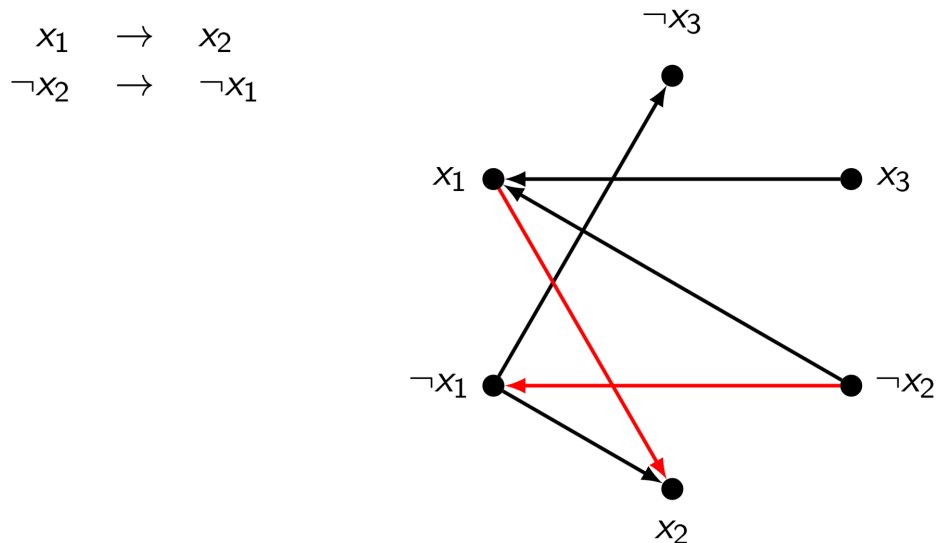
Beispiel-Reduktionen (Entscheidbarkeit & Komplexität)

1. 2-SAT $\leq$ GRAPH-ERREICHBARKEIT (Turing Reduktion)

- **2-SAT:** Erfüllbarkeit einer logischen Formel, max 2 Literale pro Klausel (z.B. $x_1 \vee \neg x_2$).
- **Idee:** Konstruiere einen "Implikationsgraphen" G_ϕ .
 - Klausel $(\alpha \vee \beta)$ ist äquivalent zu $\neg\alpha \rightarrow \beta$ und $\neg\beta \rightarrow \alpha$.
 - Knoten im Graph sind Literale $(x_i, \neg x_i)$. Kanten sind Implikationen.
- **Logik:** Formel erfüllbar $\iff$ Es gibt *keine* Variable x , sodass Pfad $x \rightarrow \dots \rightarrow \neg x$ UND Pfad $\neg x \rightarrow \dots \rightarrow x$ existiert (Widerspruchskreis).
- **Algorithmus:** Rufe für jede Variable GRAPH-ERREICHBARKEIT auf, um diese Zyklen zu prüfen.

Example

$$\phi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$



60

2. 2-FÄRBBARKEIT $\leq$ 2-SAT (Many-One Reduktion)

- **2-Färbbarkeit:** Kann man Graphen mit 2 Farben färben, sodass benachbarte Knoten ungleich gefärbt sind?
- **Reduktion:** Für jede Kante (u, v) erzeuge Klauseln: $(x_u \vee x_v) \wedge (\neg x_u \vee \neg x_v)$.
 - Bedeutet: Einer ist wahr (Farbe 1), einer falsch (Farbe 0), aber nicht beide gleich.
- Da 2-SAT effizient lösbar ist $\rightarrow$ 2-Färbbarkeit effizient lösbar.

Unentscheidbarkeitsbeweise mittels Reduktion

Um zu zeigen, dass ein neues Problem P unentscheidbar ist, reduziert man ein **bekanntes unentscheidbares Problem** (z.B. Halteproblem H) auf P .

$$H \leq P$$

Logik: Wäre P entscheidbar, wäre auch H entscheidbar (Widerspruch).

🔥 Wichtig beim Beweis

Die Reduktion muss korrekt sein: $x \in H \iff R(x) \in P$. Man zeigt beide Richtungen ($\Rightarrow$ und $\Leftarrow$).

Beispiel: KORREKTHEIT ist unentscheidbar

- **Reduktion von:** HALTEPROBLEM.
- **Gegeben:** Instanz (Π, I) (Halt).
- **Konstruiere:** (Π', I, I) für KORREKTHEIT.
 - Π' : Führt $\Pi(I)$ aus und gibt dann I zurück.
- **Argument:**
 - Hält Π auf I , dann liefert Π' den Output $I \rightarrow$ KORREKTHEIT ja.
 - Hält Π nicht, terminiert Π' nie $\rightarrow$ KORREKTHEIT nein (da Output nie geliefert wird).

Beispiel: SELBER-OUTPUT ist unentscheidbar

- **Reduktion von:** HALTEPROBLEM.
- **Konstruktion:**
 - Π_1 : `return S` (Identität, hält immer).
 - Π_2 : `call  $\Pi(S)$ ; return S` (Hält nur, wenn Π hält).
- Verhalten ist gleich $\iff \Pi$ hält (denn dann geben beide S zurück).

Beispiel: Nicht-Semi-Entscheidbarkeit zeigen

Um zu zeigen, dass P **nicht** semi-entscheidbar ist, reduziert man das **co-HALTEPROBLEM** auf P .

- Beispiel: SELBER-OUTPUT ist nicht semi-entscheidbar.
- Reduktion von co-Halt:
 - Π_1 : Endlosschleife `while(true)`.
 - Π_2 : Simuliere Π .
 - Sind beide gleich (beide Endlosschleife) $\iff \Pi$ hält *nicht*.

Berechenbarkeit

Überblick & Motivation

Recap aus Teil 1

- Es gibt **konkrete unentscheidbare Probleme** (z.B. Halteproblem).
- Unentscheidbare Probleme sind oft **semi-entscheidbar** (z.B. Gültigkeit prädikatenlogischer Formeln).
- Viele Probleme sind **nicht einmal semi-entscheidbar** (z.B. Programmäquivalenz, universelles Halteproblem).
- Die meisten Funktionen ($\mathbb{N} \rightarrow \mathbb{N}$) sind unberechenbar (überabzählbar viele Funktionen vs. abzählbar viele Programme).

Robustheit des Berechenbarkeitsmodells

Frage: Führen unterschiedliche Modelle (Turingmaschinen, λ -Kalkül, Registermaschinen) zu unterschiedlichen Begriffen von Berechenbarkeit?

Antwort: Nein. Alle hinreichend mächtigen Modelle sind äquivalent.

Church-Turing-These

Variante 1 (Turing): Wenn eine Funktion in irgendeinem konkreten Sinn berechenbar ist, so ist sie auch auf einer Turingmaschine berechenbar.

Variante 2 (Turing): Gibt es ein endlich beschreibbares Verfahren zur exakten Spezifizierung einer formalen Sprache L , so gibt es eine Turingmaschine, die L akzeptiert.

- **Zusammenhang Sprache $\leftrightarrow$ Funktion:**
 - Sprache $L \subseteq \Sigma^*$ kann als charakteristische Funktion $\chi_L : \Sigma^* \rightarrow \{0, 1\}$ betrachtet werden.
 - Funktionen $\mathbb{N}^k \rightarrow \mathbb{N}^m$ sind Mengen von Input/Output-Paaren (also Sprachen).

2.1 Historischer Kontext: Hilbert & Das Entscheidungsproblem

- **David Hilbert (1900/1928):** Wollte die Mathematik formalisieren und Widerspruchsfreiheit beweisen.
- **Entscheidungsproblem:** Suche nach einem Verfahren, das für jede prädikatenlogische Formel entscheidet, ob sie gültig ist (oder ob eine Diophantische Gleichung lösbar ist).
- **Motivation für Church & Turing:** Um zu zeigen, dass es **kein** solches Verfahren gibt (negative Lösung), brauchte man erst eine präzise mathematische Definition von "Verfahren" bzw. "Berechenbarkeit".

2.2 Imperatives Paradigma: Turingmaschinen (TM) & Registermaschinen (RM)

Turings Analyse (1936)

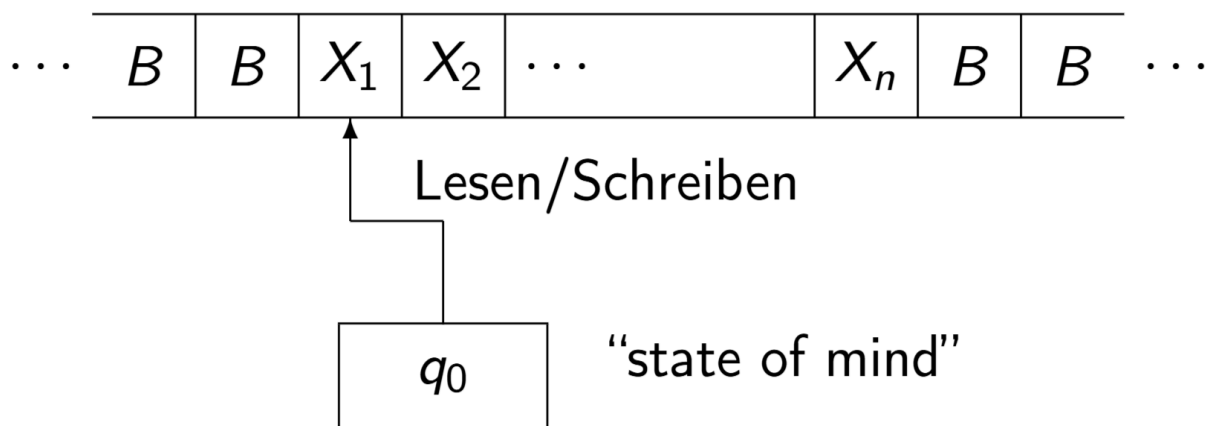
Turing analysierte, was ein menschlicher Rechner ("Computer" war damals eine Berufsbezeichnung) tut:

- Papier (2D) → Band (1D).
- Endlich viele Symbole.
- "State of mind" → Zustände.
- Diskrete Schritte.

Formale Definition einer TM

Eine Turingmaschine ist ein 7-Tupel $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$:

- Q : Endliche Menge von Zuständen.
- Σ : Eingabealphabet.
- Γ : Bandalphabet ($\Sigma \subsetneq \Gamma, B \in \Gamma$).
- δ : Übergangsfunktion $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ (Schreiben, Bewegung L/R/Stay, neuer Zustand).
- q_0 : Startzustand.
- B : Blank-Symbol.
- F : Endzustände.



Konfiguration & Berechnung

- **Konfiguration:** $X_1 \dots X_{i-1} q X_i \dots X_n$ (Bandinhalt, aktueller Zustand q , Kopfposition bei X_i).
- **Akzeptierte Sprache (rekursiv aufzählbar / semi-entscheidbar):**
 $L(M) = \{w \in \Sigma^* \mid M \text{ hält auf Eingabe } w \text{ in einem Endzustand}\}.$
- **Entscheidbar (rekursiv):** M hält *immer* (für jede Eingabe) und akzeptiert oder verwirft explizit.

Berechnung arithmetischer Funktionen

Um $f : \mathbb{N}^k \rightarrow \mathbb{N}$ zu berechnen:

1. Input $(n_1, \dots, n_k)$ wird binär auf das Band codiert.
 2. TM arbeitet.
 3. Bei Terminierung steht das Ergebnis $f(n_1, \dots, n_k)$ auf dem Band.
- Ist f undefiniert für einen Input, terminiert die TM nicht.

≡ Beispiel TM

Eine TM, die eine Binärzahl verdoppelt ($f(n) = 2n$, entspricht Anhängen einer 0) oder komplexere Operationen ausführt.

Beispiel

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, B\}, \delta, q_0, B, \{q_2\}),$$

Zustand		0	1	B
δ gegeben durch:	q_0	$(q_0, 0, R)$	$(q_0, 1, R)$	$(q_1, 0, S)$
	q_1	$(q_1, 0, L)$	$(q_1, 1, L)$	(q_2, B, R)

Beispielrechnung:

Eingabe: 6, daher auf dem Band $[6]_2 = 110$

$$\begin{aligned} q_0 110 &\Rightarrow 1 q_0 10 \Rightarrow 11 q_0 0 \Rightarrow 110 q_0 [B] \Rightarrow 110 q_1 0 \\ &\Rightarrow 11 q_1 00 \Rightarrow 1 q_1 100 \Rightarrow q_1 1100 \Rightarrow q_1 B 1100 \Rightarrow q_2 1100 \end{aligned}$$

Ausgabe: 12, weil auf dem Band $1100 = [12]_2$

Berechnete Funktion: $f_1 = F(M) : \mathbb{N} \rightarrow \mathbb{N}$, wobei $f_1(n) = 2n$

Achtung: Dieselbe TM M , angewendet auf Zahlenpaare, berechnet $f_2 = F(M) : \mathbb{N}^2 \rightarrow \mathbb{N}$, wobei $f_2(m, n) = m 2^{bl(n)+1} + n$, dabei ist bl die binäre Länge: $bl(k+1) = \lfloor \log_2(k+1) \rfloor + 1$, $bl(0) = 1$

18

Universelle Turingmaschine (UTM)

- Eine TM entspricht einem **Programm**.
- Eine UTM ist eine Maschine, die den Code $\langle M \rangle$ einer anderen TM M und eine Eingabe w nimmt und M auf w simuliert.
- **Entsprechung:** $UTM \approx$ Interpreter.

Registermaschinen (RM)

Ein Modell näher an echter Hardware (imperativ).

- **Komponenten:**
 - Speicher $S = \mathbb{N}^m$ (Register $R(1), \dots, R(m)$).

- Befehle: Addieren (A_i), Subtrahieren (S_i , modifiziert: $0 - 1 = 0$), Test auf 0 (t_i).
- **Programm:** Nummerierte Liste von Anweisungen (Sprungmarken).

≡ RM-Code Beispiele (Add/Sub)

Rechnen mit RM-Programmen

Berechnung einer Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ auf einer m -RM ($k \leq m$):

Anfangszustand: $R(i) = n_i$ für $1 \leq i \leq k$, $R(i) = 0$ für $k + 1 \leq i \leq m$

Endzustand: Endmarke erreicht, $f(n_1, \dots, n_k) = R(1)$

Beachte: Eine RM hat im Allgemeinen mehr Register als Eingabebezahlen

Konfigurationen:

$p:(n_1, \dots, n_m)$... aktuelle Kennmarke ist p ; $R(i) = n_i$ nach Ausführung der vorhergehenden Befehle, bzw. anfangs, falls p die Startmarke ist

Ein Additions-Programm:

$ADD = (0, \{(0, t_2, 5, 1), (1, S_2, 2), (2, A_1, 0)\})$

Beispielrechnung: $0:(5, 2) \Rightarrow 1:(5, 2) \Rightarrow 2:(5, 1) \Rightarrow 0:(6, 1)$
 $\Rightarrow 1:(6, 1) \Rightarrow 2:(6, 0) \Rightarrow 0:(7, 0) \Rightarrow 5:(7, 0)$

Ein Subtraktions-Programm ($\div$):

$SUB = (0, \{(0, t_2, 5, 1), (1, S_2, 2), (2, S_1, 0)\})$

Beispielrechnung: $0:(5, 2) \Rightarrow 1:(5, 2) \Rightarrow 2:(5, 1) \Rightarrow 0:(4, 1)$
 $\Rightarrow 1:(4, 1) \Rightarrow 2:(4, 0) \Rightarrow 0:(3, 0) \Rightarrow 5:(3, 0)$

23

🔗 Wichtige Fakten zu RMs

- RMs sind **berechnungsäquivalent** zu TMs.
- Bereits **2 Register** genügen für volle Universalität (durch Gödelisierung/Primfaktorzerlegung der Inhalte).
- Basis für **Church-Turing-These V3 (Shepherdson & Sturgis)**: Informell berechenbar $\Rightarrow$ auf RM berechenbar.

2.3 Funktionales Paradigma: λ -Kalkül

Idee von Alonzo Church (1936): Alles ist eine Funktion (auch Zahlen, Wahrheitswerte).

Vermeidung von Zweideutigkeiten: $\lambda x. x + 1$ (die Funktion selbst) vs. $x + 1$ (das Ergebnis).

Syntax der λ -Terme

- **Variablen:** $x, y, z, \dots$
- **Applikation:** (MN) (Wende Funktion M auf Argument N an).

- **Abstraktion:** $(\lambda x. M)$ (Funktion mit Parameter x und Rumpf M).

Konventionen:

- Linksassoziativ: $MNL = (MN)L$.
- λ -Bereich so weit wie möglich nach rechts.
- Currying: Funktionen mit mehreren Argumenten werden zu verschachtelten 1-stelligen Funktionen: $f(x, y) \rightarrow ((fx)y)$.

Auswertung (Reduktion)

1. **α -Reduktion (Umbenennung):** $\lambda x. x \rightarrow_\alpha \lambda y. y$ (Namen gebundener Variablen sind Schall und Rauch).
2. **β -Reduktion (Ausführung):** Einsetzen des Arguments.

$$(\lambda x. M)N \rightarrow_\beta M[N/x]$$

(Ersetze alle *freien* Vorkommen von x in M durch N).

Normalform & Church-Rosser

- **Redex:** Ein Term der Form $(\lambda x. M)N$ (kann reduziert werden).
- **Normalform:** Term ohne Redex (das "Ergebnis").
- **Satz von Church-Rosser:** Die Normalform ist (bis auf α -Konversion) **eindeutig**, egal in welcher Reihenfolge man reduziert.

Programmieren im λ -Kalkül

Man kann Datenstrukturen und Logik rein durch Funktionen simulieren:

- **Booleans:** $T = \lambda xy. x$, $F = \lambda xy. y$.
- **Logik:** $AND = \lambda pq. pqp$, $NOT = \lambda p. pFT$.
- **Church-Numerale (Zahlen):** Zahl n ist n -fache Anwendung einer Funktion f .

$$0 = \lambda fx. x, \quad 1 = \lambda fx. fx, \quad 2 = \lambda fx. f(fx)$$

- **Arithmetik:**

Beispiel:

$$\begin{aligned}
\text{succ } 2 &= \frac{(\lambda nfx. f(nfx)) 2}{\lambda fx. f(2fx)} \\
&\rightarrow_{\beta} \lambda fx. f(2fx) \\
&=_{\alpha} \lambda fx. f((\lambda f'x'. f'(f'x')) fx) \\
&\rightarrow_{\beta} \lambda fx. f((\lambda x'. f(fx')) x) \\
&\rightarrow_{\beta} \lambda fx. f(f(fx)) = 3
\end{aligned}$$

Beispiel:

$$\begin{aligned}
\text{add } 3 \ 2 &= \frac{(\lambda mnfx. mf(nfx)) 3 \ 2}{(\lambda nfx. 3f(nfx)) 2} \\
&\rightarrow_{\beta} \frac{(\lambda nfx. 3f(nfx)) 2}{\lambda fx. 3f(2fx)} \\
&\rightarrow_{\beta} \lambda fx. 3f(2fx) \\
&=_{\alpha} \lambda fx. 3f((\lambda f'x'. f'(f'x')) fx) \\
&\rightarrow_{\beta} \lambda fx. 3f(\lambda x'. f(fx')) x \\
&\rightarrow_{\beta} \lambda fx. 3f(f(fx)) \\
&=_{\alpha} \lambda fx. (\lambda f'x'. f'(f'(f'x')) f(fx)) \\
&\rightarrow_{\beta} \lambda fx. (\lambda x'. f(f(fx')))(f(fx)) \\
&\rightarrow_{\beta} \lambda fx. f(f(f(fx))) = 5
\end{aligned}$$

Rekursion: Der Y-Kombinator

Da λ -Funktionen keinen Namen haben, wie ruft man sich selbst auf?

Lösung: [Fixpunkt-Kombinator Y](#).

$$Y = \lambda f. (\lambda x. f(xx))(\lambda x. f(xx))$$

Eigenschaft: $YF =_{\beta} F(YF)$. Erlaubt Rekursion.

Church-Turing-These (Variante 4 - Church)

Arithmetische Funktion informell berechenbar $\iff \lambda$ -definierbar $\iff$ Turing-berechenbar.

2.4 Rekursive Funktionen (Klassischer Zugang)

Induktive Definition von Berechenbarkeit durch *Grundfunktionen* und *Operatoren*.

1. Grundfunktionen

- **Constant:** $C_n^k(x_1 \dots x_k) = n$.
- **Successor:** $S(x) = x + 1$.
- **Projection:** $P_i^k(x_1 \dots x_k) = x_i$.

2. Operatoren

- **Komposition:** Einsetzen von Funktionen ineinander. $f = h \circ (g_1, \dots, g_m)$.
- **Primitive Rekursion (Pr):**
 - Definiert f durch einen Basisfall (g) und einen Rekursionsschritt (h).
 - $f(0, \dots) = g(\dots)$
 - $f(m+1, \dots) = h(m, f(m, \dots), \dots)$
 - Funktionen, die nur hieraus bestehen, heißen **primitiv-rekursive Funktionen**. Sie sind **immer total** (terminieren immer).
 - **Beispiele:** Add, Mult, Fact, Signum, Subtraktion (abgeschnitten bei 0).

Der Operator **Pr** der primitiven Rekursion

Primitive Rekursion **Pr**

Gegeben $g : \mathbb{N}^k \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$, ist

$$f = \mathbf{Pr}(g, h) : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$$

definiert durch $f(0, x_1, \dots, x_k) = g(x_1, \dots, x_k)$ und
 $f(m+1, x_1, \dots, x_k) = h(m, f(m, x_1, \dots, x_k), x_1, \dots, x_k)$.

Die mit **Pr** und $\circ$ aus Grundfunktionen definierbaren Funktionen heißen **primitiv-rekursive Funktionen**.

Beispiel: $Add = \mathbf{Pr}(P_1^1, S \circ P_2^3)$

$$\begin{aligned} Add(0, 5) &= P_1^1(5) = 5 \\ Add(1, 5) &= S \circ P_2^3(0, 5, 5) = S(5) = 6 \\ Add(2, 5) &= S \circ P_2^3(1, 6, 5) = S(6) = 7 \\ Add(3, 5) &= S \circ P_2^3(2, 7, 5) = S(7) = 8 \end{aligned}$$

51

3. Minimierung (μ -Operator)

Um **alle** berechenbaren Funktionen zu erhalten (auch die nicht-totalen), braucht man unbeschränkte Suche.

- **μ -Operator:** μf sucht das **kleinste** y , sodass $f(y, \dots) = 0$.
- Dies führt zu **partiell rekursiven Funktionen** (äquivalent zu TM-berechenbar).
- Die Suche muss nicht terminieren $\rightarrow$ partielle Funktion.

Grenzen der primitiven Rekursion

Es gibt Funktionen, die **total** und **berechenbar** sind, aber **nicht primitiv-rekursiv**.

- Beweis durch Diagonalisierung (Aufzählung aller primitiv-rekursiven Funktionen F_i und Konstruktion $D(n) = F_n(n) + 1$).

- **Ackermann-Funktion:** Wächst extrem schnell, ist total berechenbar, aber nicht primitiv-rekursiv.

Ackermann-Funktion

Eine total rekursive, aber nicht primitiv rekursive Funktion

Ackermann-Funktion

Die Funktion $Ack : \mathbb{N}^2 \rightarrow \mathbb{N}$ ist durch folgende Gleichungen definiert:

$$\begin{aligned} Ack(0, n) &= n + 1 \\ Ack(m + 1, 0) &= Ack(m, 1) \\ Ack(m + 1, n + 1) &= Ack(m, Ack(m + 1, n)) \end{aligned}$$

Es ist **nicht ganz offensichtlich, dass Ack berechenbar ist!**

Übungsaufgabe: Berechnen Sie $Ack(3, 0)$!

Es gilt:

$$\begin{aligned} Ack(1, n) &= n + 2 \\ Ack(2, n) &= 2n + 3 \\ Ack(3, n) &= 2^{n+3} - 3 \\ Ack(4, n) &= \underbrace{2^{2^{\dots^2}}}_{n+3 \text{ Zweier}} - 3 \end{aligned}$$

Theorem

Ack **ist berechenbar und total, aber nicht primitiv rekursiv.**

59

2.5 Der Satz von Rice

Das mächtigste Werkzeug für Unentscheidbarkeit bei Sprachen/Funktionen.

Intensional vs. Extensional

- **Intensional:** Bezieht sich auf die Darstellung/den Code (Ebene 2). Z.B. "Hat die TM 5 Zustände?".
- **Extensional:** Bezieht sich auf das Verhalten/die Sprache (Ebene 1). Z.B. "Akzeptiert die TM das Wort 'Hallo?'".

Satz von Rice

Theorem

Jede **nicht-triviale** Eigenschaft $\mathcal{E}$ der rekursiv aufzählbaren Sprachen ist **unentscheidbar**.

- **Eigenschaft:** Eine Menge von Sprachen.

- **Nicht-trivial:** Es gibt mindestens eine r.a. Sprache, die die Eigenschaft hat, und eine, die sie nicht hat ($\mathcal{E} \neq \emptyset$ und $\mathcal{E} \neq$ alle r.a. Sprachen).

Analog für Funktionen: Jede nicht-triviale Eigenschaft der von TMs berechneten Funktionen ist unentscheidbar.

Anwendungen

Folgende Fragen sind **unentscheidbar** (nach Rice):

- Ist $L(M)$ endlich?
- Ist $L(M)$ leer?
- Ist $L(M) = \Sigma^*$?
- Berechnet M die Funktion $f(x) = x + 1$?
- Berechnet M eine totale Funktion?

Nicht anwendbar wenn:

- Die Eigenschaft trivial ist (trifft auf alle oder keine zu).
- Die Eigenschaft intensional ist (z.B. "Hat der Code von M mehr als 100 Zeichen?").

2.6 Weitere unentscheidbare Probleme

Busy-Beaver-Funktion $BB(n)$

- $BB(n)$ = Maximale Anzahl von 1en, die eine haltende TM mit n Zuständen (auf leerem Band) schreiben kann.
- $BB(n)$ ist eine totale Funktion, aber **nicht berechenbar** (wächst schneller als jede berechenbare Funktion).

Kolmogorov-Komplexität $K(n)$

- $K(n)$ = Länge des kürzesten Programms (auf einer UTM), das die Zahl n ausgibt.
- Maß für die "Zufälligkeit" oder Informationsdichte.
- K ist **unberechenbar**.

Postsche Korrespondenzproblem (PCP)

- Gegeben: Eine Menge von Wortpaaren (Dominosteine) $\{(v_1, w_1), \dots, (v_n, w_n)\}$.
- Frage: Gibt es eine Folge von Indizes, sodass die zusammengesetzten v 's gleich den zusammengesetzten w 's sind? ($v_{i_1} \dots v_{i_k} = w_{i_1} \dots w_{i_k}$).
- **Unentscheidbar**.

2.7 Unkonventionelle Modelle

- **Mechanisch:** Abakus, Babbage Engines (historisch).

- **Quantum Computing (QTM):**
 - Superposition & Verschränkung.
 - Theoretisch gleiches Berechenbarkeitsmaß wie TMs (Church-Turing hält).
 - Aber: Komplexitätsklasse anders (manche Probleme evtl. exponentiell schneller, aber NP-vollständig bleibt wohl schwer).
- **DNA Computing:** Massive Parallelität durch biochemische Prozesse.
- **Interactive Computing:** Modellierung von Systemen, die permanent mit der Umwelt interagieren (Web, Agenten).

Formale Sprachen

Grundlagen: Wörter und Sprachen

Grundbegriffe

- **Alphabet (Σ):** Endliche, nicht-leere Menge atomarer Symbole (z.B. $\Sigma = \{a, b\}$).
- **Wort (w):** Endliche Folge von Symbolen aus Σ .
- **Länge ($|w|$):** Anzahl der Symbole in w .
- **Leerwort (ϵ):** Wort der Länge 0 ($|\epsilon| = 0$).
- **Häufigkeit ($|w|_a$):** Anzahl des Symbols a in Wort w .

Operationen auf Wörtern

- **Konkatenation:** Hintereinanderschreiben ($x \cdot y = xy$). $|xy| = |x| + |y|$.
- **Potenzbildung:**
 - $w^0 = \epsilon$
 - $w^n = ww^{n-1}$
- **Spiegelbild (w^r):** Wenn $w = a_1 \dots a_n$, dann $w^r = a_n \dots a_1$.
 - **Palindrom:** Wenn $w = w^r$.

Formale Sprachen

- **Formale Sprache L :** Beliebige Teilmenge $L \subseteq \Sigma^*$.
- Σ^+ : Menge aller nicht-leeren Wörter.
- Σ^* : Menge aller Wörter (inkl. ϵ). $(\Sigma^*, \cdot, \epsilon)$ bildet ein Monoid.
- **Potenzmenge $\mathcal{P}(\Sigma^*)$:** Menge aller Sprachen.

Beispiele:

- $L = \{a^n b^n \mid n \geq 0\}$ (Gleich viele a's wie b's, sortiert).
- $L = \{w \in \{0, 1\}^* \mid w = w^r\}$ (Palindrome).

Operationen auf Sprachen

Seien L, L_1, L_2 Sprachen.

1. Mengenoperationen:

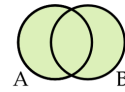
- **Vereinigung:** $L_1 \cup L_2$
- **Durchschnitt:** $L_1 \cap L_2$
- **Differenz:** $L_1 - L_2$
- **Komplement:** $\bar{L} = \Sigma^* - L$ (Alles was nicht in L ist).

Visualisierung Mengenoperationen

Mengenoperationen auf Sprachen

Vereinigung:

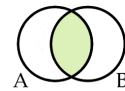
$$A \cup B = \{x \in \Sigma^* \mid x \in A \text{ oder } x \in B\}$$



Durchschnitt:

$$A \cap B = \{x \in \Sigma^* \mid x \in A \text{ und } x \in B\}$$

(A und B sind **disjunkt** wenn $A \cap B = \{\}$)



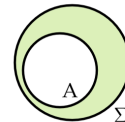
Differenz:

$$A - B = \{x \in \Sigma^* \mid x \in A \text{ und } x \notin B\}$$



Komplement:

$$\overline{A} = \Sigma^* - A = \{x \in \Sigma^* \mid x \notin A\}$$



$A \subseteq B$: A ist eine **Teilmenge** von B (jedes Element von A ist auch ein Element von B)

$A \subset B$: A ist eine **echte Teilmenge** von B (A ist eine Teilmenge von B, die nicht gleich B ist)

$A = B$ wenn $A \subseteq B$ und $B \subseteq A$

7

2. Sprachspezifische Operationen:

- **Konkatenation:** $L_1 \cdot L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$.
- **Potenz:** $L^0 = \{\epsilon\}$, $L^{n+1} = L L^n$.
- **Kleene-Stern (L^*):** Beliebige (auch 0-malige) Verkettung von Wörtern aus L . $L^* = \bigcup_{n \geq 0} L^n$. Enthält immer ϵ .
- **Plus (L^+):** Mindestens einmalige Verkettung. $L^+ = \bigcup_{n \geq 1} L^n$. ($L^+ = L^* \cdot L$).

Algebraische Struktur

$(\mathcal{P}(\Sigma^*), \cup, \cdot, \emptyset, \{\epsilon\})$ bildet einen **nichtkommutativen Semiring**.

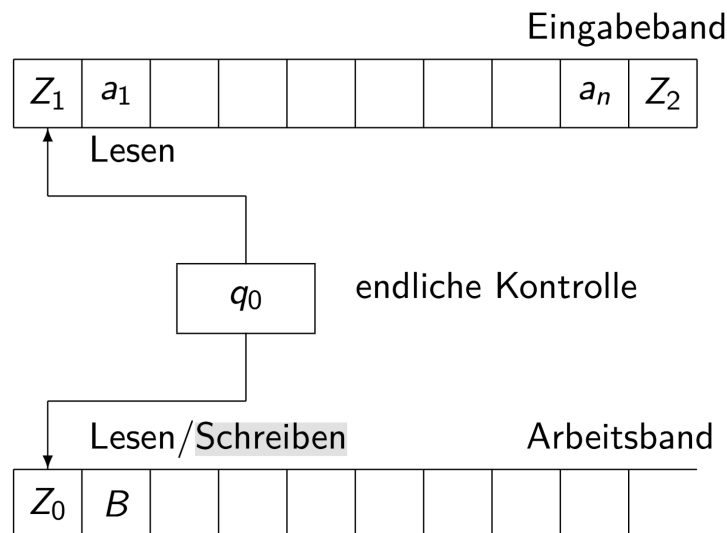
- Wichtiges Gesetz: $A \cdot (B \cup C) = A \cdot B \cup A \cdot C$ (Distributivität).

Turingmaschinen (TM) & Automatenmodelle

Ein zentrales Konzept der theoretischen Informatik zur Klassifizierung von Sprachen.

Aufbau einer Turingmaschine

Turingmaschinen² revisited



²Alan M. Turing (1912 - 1954)

Definition der TM

Eine TM ist ein 8-Tupel $M = (Q, T, \Gamma, \delta, q_0, \{Z_0, Z_1, Z_2\}, B, F)$.

- **Bänder:** Eingabeband (read-only, begrenzt durch Z_1, Z_2) & Arbeitsband (read/write, unendlich).
- **Kopfbewegungen:** L (Links), R (Rechts), S (Stay/Stehenbleiben).
- **Übergangsfunktion δ :** $(q, a, X) \rightarrow (p, Y, D_E, D_A)$.
 - Liest Symbol a (Eingabe) und X (Arbeit).
 - Wechselt zu Zustand p .
 - Schreibt Y auf Arbeitsband.
 - Bewegt Köpfe (D_E, D_A) .

Akzeptanz: Ein Wort wird akzeptiert, wenn die TM in einen Endzustand $q \in F$ gelangt.

Varianten & Normalformen

- **Deterministisch:** Für jede Situation höchstens ein Übergang möglich.
- **Normalform:** Genau ein Endzustand, leeres Arbeitsband am Ende.
- **Linear Beschränkter Automat (LBA):** Arbeitsband-Platz ist linear zur Eingabelänge beschränkt ($cn + d$). $\rightarrow$ **Kontextsensitive Sprachen**.
- **Kellerautomat (Pushdown Automaton - PDA):**
 - Arbeitsband ist ein **Stack (Keller)** (LIFO - Last In First Out).
 - Darf nur oberstes Symbol lesen/schreiben.

- Eingabeband nur R (rechts) oder S (stehen).
 - → **Kontextfreie Sprachen**.
 - **Endlicher Automat (EA):**
 - Kein Arbeitsband. Nur Zustandswechsel.
 - Bewegung auf Eingabe nur rechts (R).
 - → **Reguläre Sprachen**.
-

Reguläre Sprachen (Typ 3)

Die einfachste Sprachklasse.

Definition: Die Menge der regulären Sprachen $\mathcal{L}_{reg}(\Sigma)$ ist die kleinste Menge, die:

1. $\emptyset$ und $\{a\}$ (für alle $a \in \Sigma$) enthält.
2. Abgeschlossen ist unter Vereinigung ($\cup$), Konkatenation ($\cdot$) und Stern ($*$).

Reguläre Ausdrücke

Algebraische Notation für reguläre Sprachen.

- Priorität: $*$ (hoch) $>$ $\cdot$ (mittel) $>$ $+$ (niedrig, Vereinigung).
- Beispiel: $\{01\}^* \neq \{0, 1\}^*$.
- Achtung: In Slides wird $+$ oft für Vereinigung genutzt ($L_1 + L_2$ statt $L_1 \cup L_2$).

Endliche Automaten (EA)

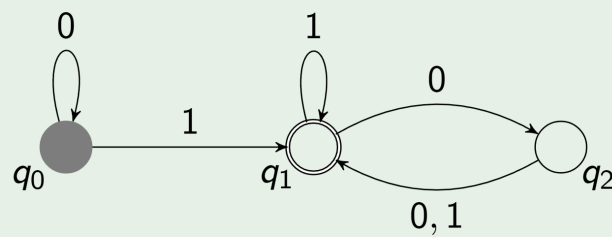
Modell mit endlichem Speicher (Zustände).

Deterministischer Endlicher Automat (DEA)

- **5-Tupel:** $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$.
- $\delta : Q \times \Sigma \rightarrow Q$ ist eine **totale** Funktion (zu jedem Paar gibt es genau einen Folgezustand).
- **Falle:** Ein Zustand, der nicht mehr verlassen werden kann und kein Endzustand ist (für "ungültige" Eingaben).
- **Sprache:** $\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$.

☰ Beispiel DEA

DEA: Beispiel



Zustände: q_0, q_1, q_2

Startzustand:  (oder auch )

Endzustand: 

Transitionen (Übergänge): Pfeile

Eingabe: Wort; Ausgehend vom Startzustand liest der Automat $\mathcal{A}$ von links nach rechts Symbol für Symbol. Nach dem Lesen eines Symbols geht $\mathcal{A}$ in den nächsten Zustand über indem er entlang der mit diesem Symbol markierten Kante geht. Nach dem Lesen des letzten Symbols wird der "Output" erzeugt: Befindet sich $\mathcal{A}$ in einem Endzustand, wird das Wort akzeptiert; ansonsten wird das Wort nicht akzeptiert.

29

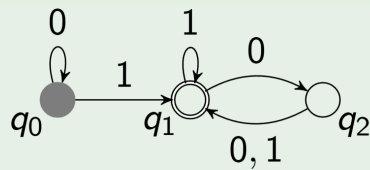
Nichtdeterministischer Endlicher Automat (NEA)

- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ (Mehrere oder gar keine Folgezustände möglich).
- **ϵ -NEA:** Erlaubt Übergänge ohne Inputzeichen (ϵ -Übergänge).
- **Akzeptanz:** Es existiert *mindestens ein* Pfad, der in einen Endzustand führt.

Vergleich DEA vs NEA

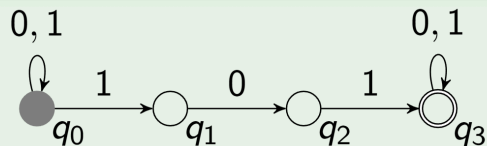
Nichtdeterministische endliche Automaten (NEA)

DEA



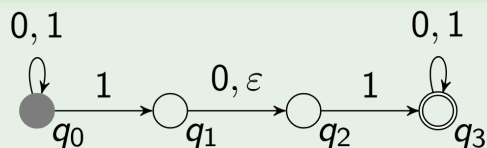
Von einem Zustand aus gibt es mit ein und demselben Eingabesymbol genau einen Folgezustand.

NEA



Von einem Zustand aus kann es mit ein und demselben Eingabesymbol mehrere Folgezustände geben.

ϵ -NEA



Von einem Zustand aus kann es mit ein und demselben Eingabesymbol mehrere Folgezustände geben, Übergänge sind auch mit ϵ möglich (ϵ -Übergänge).

34

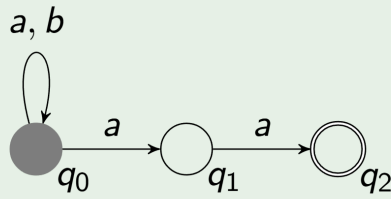
Umwandlung & Äquivalenz

1. **Potenzmengenkonstruktion (Subset Construction):** Wandelt jeden NEA in einen äquivalenten DEA um.
 - Zustände des DEA sind Teilmengen der Zustände des NEA ($2^{|Q|}$).
 - Startzustand: $\{q_0\}$ (bzw. Hülle bei ϵ -NEA).
 - Endzustände: Alle Mengen, die mind. einen Endzustand des NEA enthalten.
2. **Minimierung (Brzowski):** Zu jeder regulären Sprache gibt es einen eindeutigen **Minimalautomaten**.
 - Algorithmus: Reverse $\rightarrow$ Determinize $\rightarrow$ Reverse $\rightarrow$ Determinize.

Potenzmengenkonstruktion Beispiel

Determinisierung: Beispiel

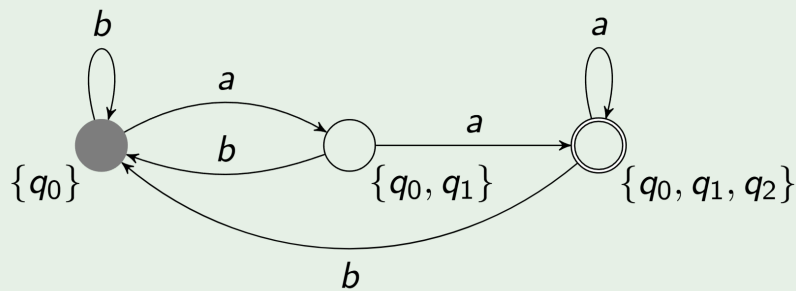
NEA:



δ	a	b
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\{\}$
q_2	$\{\}$	$\{\}$

$\hat{\delta}$	a	b
SZ $\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$
EZ $\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0\}$

DEA:



37

Entscheidungsprobleme & Eigenschaften

- **Entscheidbar:** Wortproblem ($w \in L?$), Leerheit, Endlichkeit, Äquivalenz ($L_1 = L_2?$).
- **Abgeschlossenheit:** $\cup, \cdot, *,$ Komplement, $\cap, \setminus,$ Spiegelung, Homomorphismus.

Pumping Lemma für reguläre Sprachen

Dient dazu zu zeigen, dass eine Sprache **nicht** regulär ist.

Pumping Lemma

Sei L regulär und unendlich. Dann $\exists m > 0$, sodass $\forall w \in L$ mit $|w| \geq m$ eine Zerlegung $w = xyz$ existiert mit:

1. $|xy| \leq m$
2. $|y| > 0$ (der pumpbare Teil ist nicht leer)
3. $\forall i \geq 0 : xy^iz \in L$

Beweisidee (Schubfachprinzip): Wenn das Wort mehr Zeichen hat als der Automat Zustände (m), muss sich ein Zustand wiederholen (Zyklus). Dieser Zyklus (y) kann beliebig oft durchlaufen werden.

Beispiel für NICHT regulär: $L = \{a^n b^n | n \geq 0\}$. Man kann beim "Aufpumpen" die gleiche Anzahl von a und b nicht garantieren.

Grammatiken

Allgemeines Erzeugungssystem für Sprachen.

Definition: $G = (N, T, P, S)$

- N : Nichtterminale (Variablen, z.B. S, A).
- T : Terminale (Alphabet, z.B. a, b).
- P : Produktionen (Regeln $\alpha \rightarrow \beta$).
- S : Startsymbol.

Ableitung ($\Rightarrow$): Ersetzen von Variablen durch rechte Regelseiten.

Satzform: Wort, das noch Variablen enthalten kann.

Erzeugte Sprache $\mathcal{L}(G)$: Alle Wörter aus T^* , die aus S ableitbar sind.

Chomsky-Hierarchie (Grammatiken)

Typ	Name	Regelform ($\alpha \rightarrow \beta$)	Automat
Typ 0	Rekursiv aufzählbar	Beliebig ($\alpha \in (N \cup T)^+$)	Turingmaschine
Typ 1	Kontextsensitiv (Monoton)	S	\alpha
Typ 2	Kontextfrei	$A \rightarrow \beta \ (A \in N)$	Kellerautomat
Typ 3	Regulär	$A \rightarrow aB$ oder $A \rightarrow \epsilon$	Endlicher Automat

- **Anmerkung:** Bei Typ 1 ist $S \rightarrow \epsilon$ erlaubt, wenn S rechts nicht vorkommt.

Kontextfreie Sprachen (Typ 2)

Wichtig für Programmiersprachen (Syntax, Klammerung).

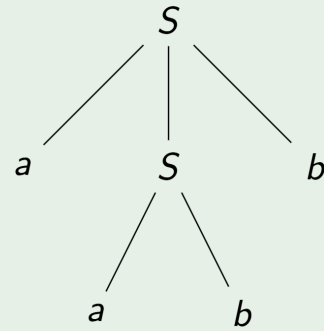
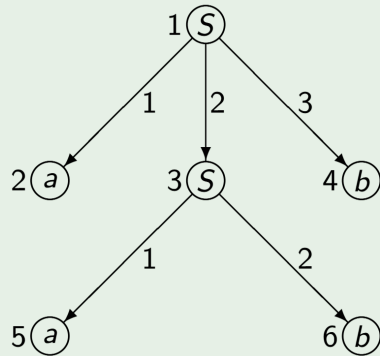
- Produktionen: $A \rightarrow \beta$ (Links steht immer genau ein Nichtterminal).
- **Ableitungsbaum:** Grafische Darstellung der Ableitung. Wurzel = S , Blätter = Terminale (Wort).
- **Mehrdeutigkeit:** Eine Grammatik ist mehrdeutig, wenn es für ein Wort **mehrere verschiedene Ableitungsbäume** (bzw. Linksableitungen) gibt.
 - **Inhärent mehrdeutig:** Die Sprache selbst lässt nur mehrdeutige Grammatiken zu.

≡ **Ableitungsbaum**

Ableitungsbäume: Beispiel

$G = (\{S\}, \{a, b\}, \{S \rightarrow aSb, S \rightarrow ab\}, S)$

Ableitungsbaum für $aabb$:



Front des Ableitungsbaums:

ergibt sich als Folge der Labels der Knoten 2, 5, 6, 4 zu $aabb$

73

Normalformen (für Typ 2)

1. **Reduzierte Grammatik:** Keine nutzlosen Symbole, keine ϵ -Produktionen (außer $S \rightarrow \epsilon$), keine Kettenregeln ($A \rightarrow B$).
2. **Chomsky Normalform (CNF):**
 - $A \rightarrow BC$
 - $A \rightarrow a$
 - Erlaubt binäre Ableitungsbäume $\rightarrow$ wichtig für Pumping Lemma.
3. **Greibach Normalform (GNF):**
 - $A \rightarrow aw$ ($w \in N^*$).
 - Jeder Schritt produziert ein Terminal $\rightarrow$ wichtig für Parsing.

Pumping Lemma für kontextfreie Sprachen

Zeigt, dass Sprachen wie $a^n b^n c^n$ **nicht** kontextfrei sind.

Pumping Lemma (KF)

Sei L kontextfrei. $\exists m > 0$, sodass $\forall w \in L$ mit $|w| \geq m$ eine Zerlegung $w = uvxyz$ existiert mit:

1. $|vxy| \leq m$
2. $|vy| \geq 1$ (Mindestens v oder y nicht leer)

$$3. \forall i \geq 0 : uv^i xy^i z \in L$$

Idee: Wiederholung eines Nichtterminals auf einem Pfad im Ableitungsbaum (da Baumhöhe begrenzt durch $|N|$). Man kann den Teilbaum unter dem wiederholten Nichtterminal austauschen ("pumpen").

Pumping Lemma Visualisierung

Pumping Lemma für kontextfreie Sprachen - Beweis ctd.

Beweis ctd.

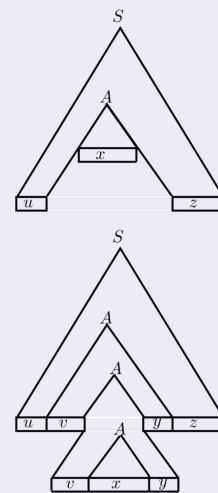
Der Ableitungsbaum für w kann auf verschiedene Arten modifiziert werden:

Beispielsweise kann an das obere A der Ableitungsbaum des unteren A gehängt werden.

Wir erhalten damit eine Ableitung von $uxz = uv^0 xy^0 z$. Das heißt, $uv^0 xy^0 z \in L(G)$.

Man kann auch an das untere A den Teilbaum anhängen, der am oberen A beginnt. Dies ergibt eine Ableitung von $uvvxyyz = uv^2 xy^2 z$.

Allgemein gilt, dass für jedes $i \geq 0$ $uv^i xy^i z \in L(G)$. □



93

Eigenschaften & Abschluss (Typ 2)

- **Abgeschlossen unter:** $\cup, \cdot, *, \text{Homomorphismus}, \cap \text{Regulär}$.
- **NICHT abgeschlossen unter:** $\cap$ (Schnitt), Komplement.
 - *Gegenbeispiel Schnitt:* $\{a^n b^n c^*\} \cap \{a^* b^n c^n\} = \{a^n b^n c^n\}$ (nicht kf).
- **Satz von Chomsky-Schützenberger:** Jede KF-Sprache ist homomorphes Bild des Schnitts einer Dyck-Sprache (Klammern) und einer regulären Sprache.
- **Entscheidbar:** Wortproblem (CYK-Algorithmus), Leerheit, Endlichkeit.
- **Unentscheidbar:** Äquivalenz, $L = \Sigma^*$, Schnitt-Leerheit.

Spezialfall unäres Alphabet

Jede kontextfreie Sprache über einem einelementigen Alphabet ($\{a\}^*$) ist **regulär**.

Jenseits von Kontextfrei (Typ 1 & 0)

Typ 1: Kontextsensitiv / Monoton

- Regeln: $uAv \rightarrow u\beta v$ (Kontext u, v bleibt, A wird ersetzt) bzw. $|\alpha| \leq |\beta|$.
- Akzeptiert durch **LBA** (Linear Bounded Automaton).
- Abschluss: Unter allen gängigen Operationen (inkl. Schnitt, Komplement).
- Beispiel: $\{a^n b^n c^n\}$ ist Typ 1.

Typ 0: Rekursiv aufzählbar

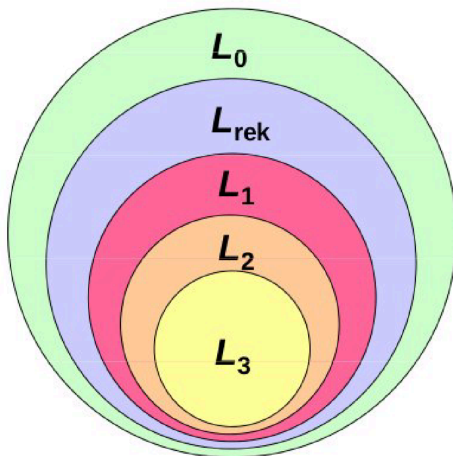
- Keine Einschränkungen an Produktionen.
- Akzeptiert durch **Turingmaschine**.
- **Rekursiv (Entscheidbar)**: Wenn L und $\bar{L}$ Typ 0 sind (TM hält immer).
 - $L_{rec} \subset L_0$.

Zusammenfassung Chomsky-Hierarchie & Eigenschaften

Hierarchie Inklusion

$$L_3 \subset L_2 \subset L_1 \subset L_{rec} \subset L_0$$

nicht rekursiv aufzählbare Sprachen



Abschlusseigenschaften Übersicht

Tabelle der Abschlusseigenschaften

	$\mathcal{L}_3$	$\mathcal{L}_2$	$\mathcal{L}_1$	$\mathcal{L}_0$
Vereinigung	ja	ja	ja	ja
Konkatenation	ja	ja	ja	ja
Kleenescher Stern	ja	ja	ja	ja
Komplement	ja	nein	ja	nein
Durchschnitt	ja	nein	ja	ja
Durchschnitt mit reg. Mengen	ja	ja	ja	ja
Homomorphismen	ja	ja	nein	ja
ε -freie Homomorphismen	ja	ja	ja	ja
<i>gsm</i> -Abbildungen	ja	ja	nein	ja
ε -freie <i>gsm</i> -Abbildungen	ja	ja	ja	ja

Hinweis zur Tabelle:

- **Homomorphismen:** $h(w)$ ersetzt Zeichen durch Wörter.
- ***gsm*-Abbildung:** Endlicher Automat mit Ausgabe (verallgemeinerter Homomorphismus). Typ 1 ist nur unter ε -freien *gsm*/Homomorphismen abgeschlossen.

Komplexitätstheorie

4.1 Grundlegende Konzepte und Wiederholung

Entscheidungsprobleme

Ein **Problem** besteht aus einer unendlichen Menge möglicher Instanzen (Inputs) und einer Frage.

- **Entscheidungsproblem:** Ein Problem, dessen Frage eine Ja/Nein-Antwort erwartet.

☰ Beispiel: GRAPH-ERREICHBARKEIT

- **Instanz:** Ein Graph $G = (V, E)$ und Knoten $u, v \in V$.
- **Frage:** Gibt es einen Pfad von u nach v ?

Algorithmen und SIMPLE

Ein Algorithmus ist eine Beschreibung von Rechenschritten, um jede Instanz eines Problems zu lösen.

- Muss nach endlich vielen Schritten terminieren.
- Muss die korrekte Antwort liefern.

Programmiersprache SIMPLE:

- Dient als Modell für prozedurale Sprachen (Variablen, Zuweisungen, `if/then/else`, `while`, `for`, `return`).
- Input: Liste von Werten oder ein großer String.

📌 Church-Turing These

- **Informell:** Jeder Algorithmus lässt sich mittels SIMPLE realisieren.
- **Erweiterte These:** Jedes "angemessene" formale Modell ist bzgl. Zeit/Speicher äquivalent zu SIMPLE (meist bis auf polynomiellen Overhead).
- **Konsequenz:** Für die Analyse von "effizient" vs. "nicht effizient" reicht die Betrachtung von SIMPLE.

Reduktionen

Eine Methode, um Probleme aufeinander abzubilden. Wir schreiben $A \leq B$ (Problem A wird auf B reduziert).

1. **Lösbarkeit (Idee 1):** Um A zu lösen, formen wir es in B um und nutzen den Algorithmus für B . (A ist höchstens so schwer wie B).
2. **Härte (Idee 2):** Wenn wir wissen, dass A schwer ist und $A \leq B$ gilt, dann muss B mindestens so schwer wie A sein (sonst könnten wir A ja über B leicht lösen).

📄 "Many-One" Reduktion (Karp Reduktion)

Eine Funktion R bildet Instanzen von A auf Instanzen von B ab.

- x ist positive Instanz von $A \Leftrightarrow R(x)$ ist positive Instanz von B .
- **Ressourcen:** R muss effizient berechenbar sein (in **polynomieller Zeit**).

Aussagenlogik (Wiederholung)

- **Syntax:** Boole'sche Variablen (X), Konnektive ($\vee, \wedge, \neg$).
- **Literal:** Variable (x) oder Negation ($\neg x$).
- **Klausel:** Disjunktion von Literalen (z.B. $l_1 \vee l_2 \vee l_3$).
- **KNF (Konjunktive Normalform):** Konjunktion von Klauseln.
- **3-KNF:** Jede Klausel hat genau 3 Literale.

Wichtige Entscheidungsprobleme:

- **MODEL-CHECKING:** Ist Formel ϕ wahr unter Belegung I ?
- **SAT (Satisfiability):** Ist ϕ erfüllbar (gibt es ein I , das ϕ wahr macht)?
- **3-SAT:** Ist eine 3-KNF Formel erfüllbar?

≡ Reduktion: 3-COL auf SAT

Wir wollen zeigen, dass Graphenfärbung (3 – COL) auf SAT reduziert werden kann.

- **Idee:** Wir bauen eine Formel, die nur wahr ist, wenn der Graph färbbar ist.
- **Atome:** a_i^c (Knoten i hat Farbe $c \in \{r, g, b\}$).
- **Klauseln:**
 1. Jeder Knoten hat mind. eine Farbe: $\bigvee_c a_i^c$
 2. Kein Knoten hat mehr als eine Farbe (Konsistenz).
 3. Nachbarn haben unterschiedliche Farben: $\neg(a_i^c \wedge a_j^c)$ für Kanten $[v_i, v_j]$.
- **Ergebnis:** G ist 3-färbbar $\Leftrightarrow \phi_G$ ist erfüllbar.

4.2 Die Komplexitätsklassen P und NP

Komplexitätsmaße

Wir betrachten die **asymptotische Worst-Case Komplexität**.

- **Laufzeit:** Anzahl atomarer Operationen als Funktion $f(n)$ der Instanzgröße n .
- **Speicherbedarf:** Anzahl der benötigten Bits im Speicher.
- **O-Notation:** $g(n) = O(f(n))$, wenn g maximal so schnell wächst wie f .

Die Klasse P (Polynomial Time)

Entscheidungsprobleme, die in polynomieller Zeit lösbar sind.

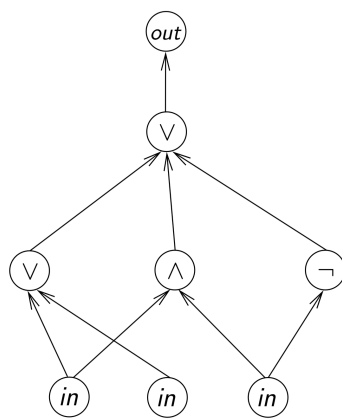
- Laufzeit beschränkt durch $O(|I|^k)$ (für Konstante k).

- Gilt als **effizient lösbar** ("tractable").
- **Beispiele:** GRAPH-ERREICHBARKEIT, 2-SAT, Kontextfreie Grammatiken, Primzahltest.

≡ Problem in P: CIRCUIT-EVAL

- **Instanz:** Boolean Circuit (Logikgatter-Schaltung) + Input-Werte.
- **Frage:** Ist der Output `true`?
- **Lösung:** Werte Gatter topologisch sortiert aus. Laufzeit linear/polynomiell zur Größe des Circuits.
- **Folgerung:** Auch **MODEL-CHECKING** ist in P (Reduktion auf CIRCUIT-EVAL).

Beispiel für einen Circuit



33

Beispiel eines Boolean Circuits

Die Klasse NP (Non-deterministic Polynomial Time)

Probleme, für die (vermutlich) kein effizienter Algorithmus existiert, aber Lösungen leicht überprüft werden können.

Definition 1: Zertifikate

Ein Problem ist in NP, wenn:

1. Positive Instanzen kompakte **Zertifikate** (Lösungen/Zeugen) haben (Größe polynomiell zur Instanz).
2. Es einfach (**in P**) ist, zu prüfen, ob ein Zertifikat gültig ist.

Zertifikat-Relation

Relation $R \subseteq \text{Instanzen} \times \text{Strings}$.

I ist positive Instanz $\Leftrightarrow \exists C : (I, C) \in R$.

- **Polynomiell balanziert:** $|C| \leq |I|^k$.
- **Polynomiell entscheidbar:** $(I, C) \in R$ ist in P prüfbar.

Definition 2: Nicht-Determinismus (Guess & Check)

Erweiterung von SIMPLE um `choice(a, b)`.

- Ein Programm teilt sich in zwei parallele Zweige.
- Programm liefert `true`, wenn **mindestens ein** Zweig `true` liefert.
- NP = Probleme, die von einem solchen Programm in polynomieller Zeit gelöst werden können.

Satz: SAT ist in NP

- **Zertifikat:** Eine Wahrheitsbelegung.
- **Check:** Prüfen der Belegung (Model-Checking) ist in P.
- Alternativ: "Rate" eine Belegung (Guess) und prüfe sie (Check).

4.3 NP-Vollständigkeit

Begriffe:

- **C-hart:** Jedes Problem aus Klasse C kann auf P reduziert werden.
- **C-vollständig:** P ist in C **und** P ist C-hart.

NP-vollständige Probleme sind die "schwersten" Probleme in NP.

- Wenn ein NP-vollständiges Problem effizient lösbar wäre (P), dann wären alle NP-Probleme effizient lösbar ($P = NP$).
- Aktueller Konsens: $P \neq NP$ (d.h. NP-vollständige Probleme sind **intractable**).

i Cook-Levin Theorem

SAT ist NP-vollständig.

- Dies ist der Startpunkt für alle weiteren Härte-Beweise.
- Beweisidee: Man kodiert die Berechnung einer Turing-Maschine (oder SIMPLE-Programm) in eine aussagenlogische Formel.

Beweisschema für NP-Vollständigkeit

Um zu zeigen, dass ein neues Problem B NP-vollständig ist:

1. Zeige $B \in NP$ (z.B. durch Angabe eines Zertifikats).
2. Wähle ein bekanntes NP-hartes Problem A (z.B. 3-SAT).
3. Zeige $A \leq_p B$ (Polynomielle Reduktion).

Wichtige NP-vollständige Probleme:

- **3-SAT:** Spezialfall von SAT (Klauseln der Länge 3). Oft einfacher für Reduktionen als SAT.
- **INDEPENDENT SET:** Gibt es K Knoten im Graphen, die paarweise nicht verbunden sind?

- **CLIQUE:** Gibt es K Knoten, die alle miteinander verbunden sind?
- **VERTEX COVER:** Gibt es K Knoten, die alle Kanten "berühren"?

Reduktion: 3-SAT auf INDEPENDENT SET

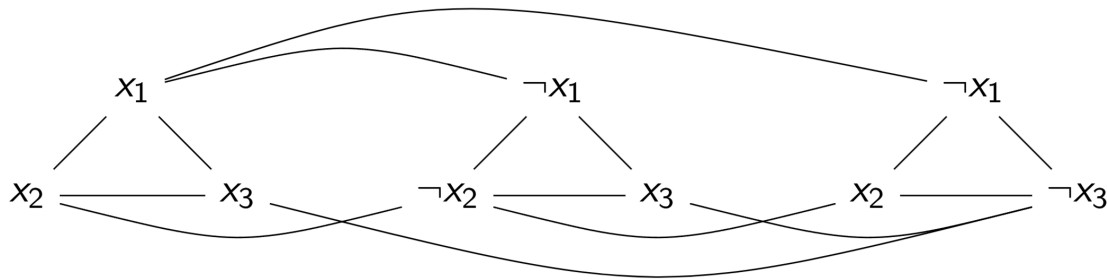
Wir konstruieren aus einer Formel ϕ einen Graphen G :

1. Für jede Klausel ein Dreieck aus 3 Knoten (Knoten = Literale).
2. Verbinde Literale, die sich widersprechen (z.B. x_1 und $\neg x_1$) mit einer Kante.
3. Setze $K = m$ (Anzahl der Klauseln).

Logik: Ein Independent Set der Größe m muss aus jedem Dreieck genau einen Knoten wählen (d.h. ein Literal pro Klausel ist wahr) und darf keine widersprüchlichen Literale wählen.

Beispiel

Die 3-KNF Formel $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$ wird auf folgenden Graph reduziert:



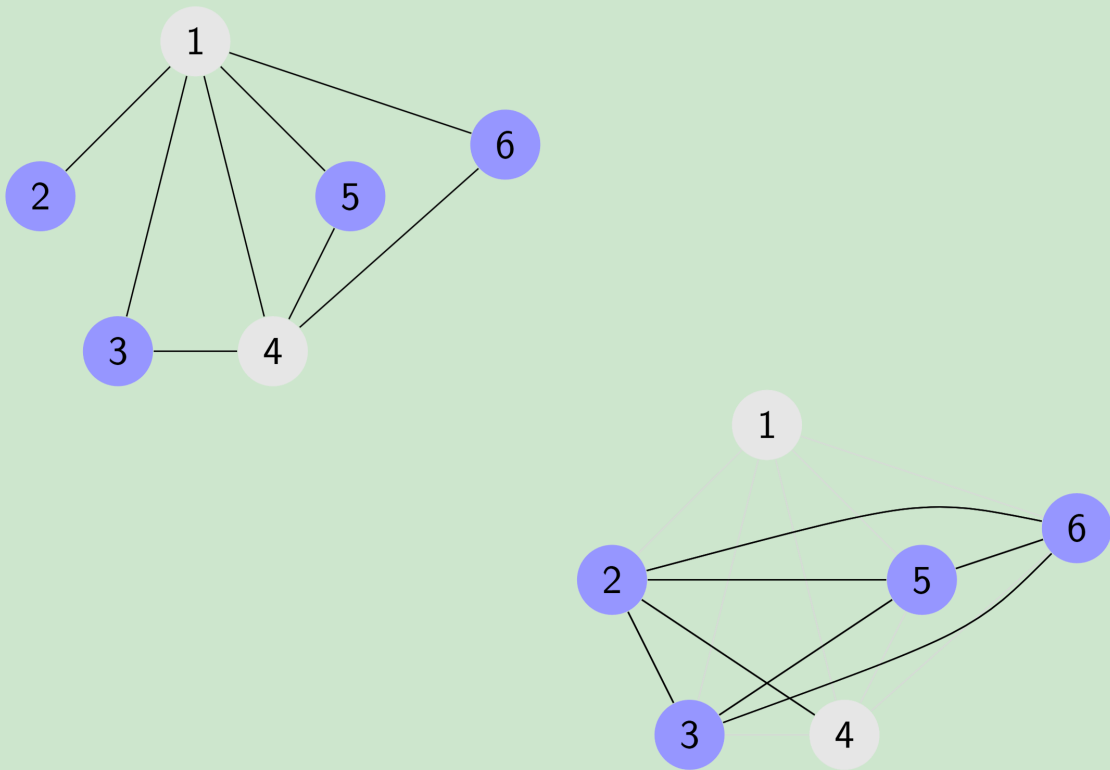
Visualisierung der Reduktion: Dreiecke für Klauseln, Kanten zwischen Widersprüchen.

Zusammenhang Graph-Probleme

- **Clique vs. Independent Set:**
 - Clique in $G \Leftrightarrow$ Independent Set im Komplementgraphen $\bar{G}$.
- **Independent Set vs. Vertex Cover:**
 - S ist Independent Set $\Leftrightarrow V \setminus S$ ist Vertex Cover.

INDEPENDENT SET vs. CLIQUE

Example

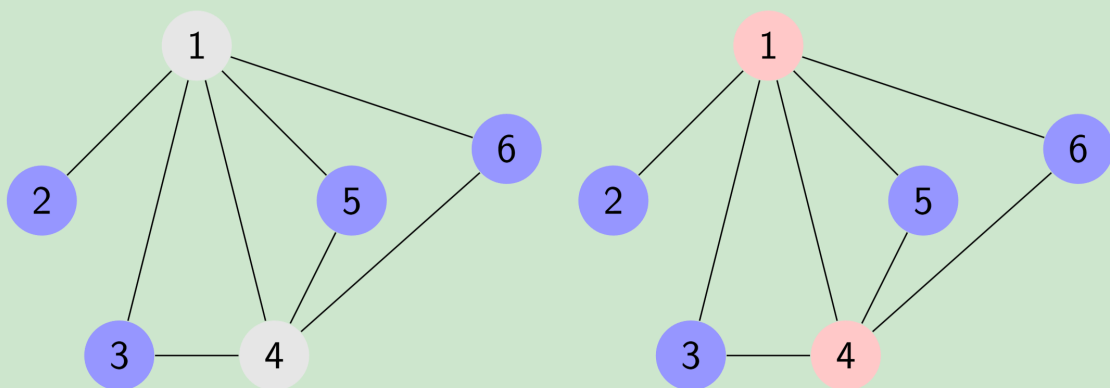


72

Links: IS (Knoten 2, 5). Rechts: Clique im Komplementgraphen.

INDEPENDENT SET vs. VERTEX COVER

Example



Links: IS (Knoten 2, 5). Rechts: Vertex Cover (Restliche Knoten).

Weitere NP-vollständige Probleme:

- **Graphen:** HAMILTON-PATH, HAMILTON-CYCLE, k -COL ($k \geq 3$).
- **Logik:** NAESAT (Not-all-equal), 1-IN-3-SAT.

⚠ Spezialfälle

Nicht jeder Spezialfall eines NP-vollständigen Problems ist einfach.

- **3-COL-NT:** 3-Färbbarkeit auf "non-terminal" Graphen (jeder Knoten Grad ≥ 2) ist weiterhin NP-vollständig.

4.4 Weitere Komplexitätsklassen

Klasse L (Logarithmic Space)

Probleme, die mit **logarithmischem Speicher** $O(\log |I|)$ lösbar sind.

- Nur konstant viele Pointer/Zähler erlaubt.
- Input liegt im Read-only Speicher (zählt nicht zum Limit).
- Laufzeit maximal polynomiell.
- $L \subseteq P$.
- **Beispiel:** FIND-NODE (Knotensuche im Baum).

Klasse PSPACE (Polynomial Space)

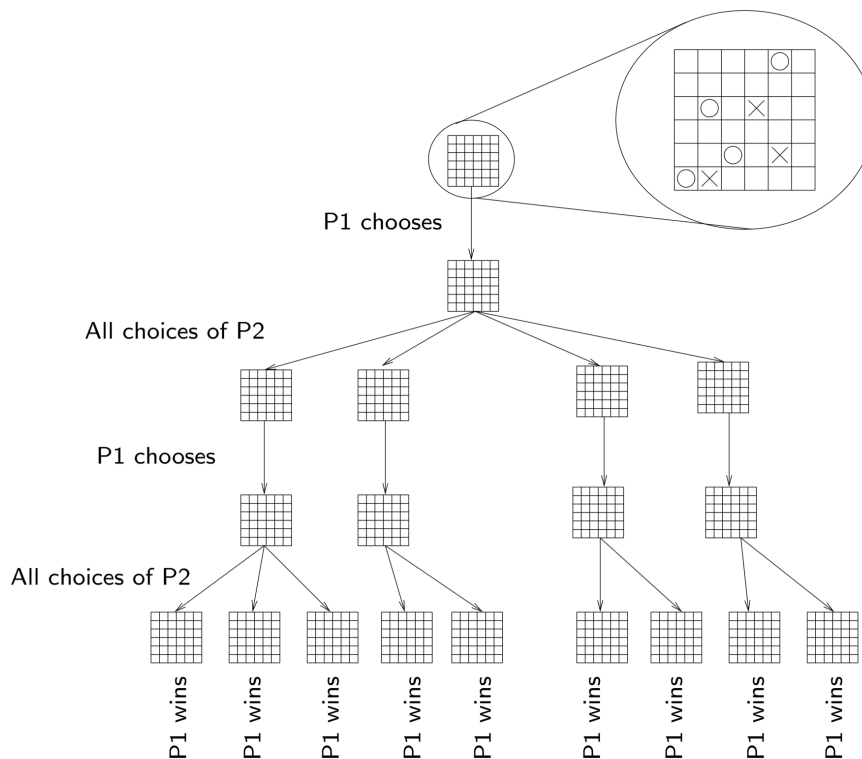
Probleme, die mit **polynomielltem Speicher** $O(|I|^k)$ lösbar sind.

- Laufzeit darf exponentiell sein.
- $NP \subseteq PSPACE$.
- Umfasst viele Spiele-Probleme ("Winning Strategy").

≡ (n,k)-Tic-Tac-Toe

Ist PSPACE-vollständig. Man kann den gesamten Spielbaum (Tiefe n^2) mit rekursiver Suche (Stack-Tiefe polynomiell) durchsuchen.

Gewinnstrategie für Verallgemeinertes Tic-Tac-Toe



92

Klasse EXPTIME

Probleme, die in exponentieller Zeit $O(2^{|I|^k})$ lösbar sind.

- $P \subsetneq EXPTIME$ (bewiesene echte Inklusion).
- Beispiele: Go, Schach (verallgemeinert).

Hierarchie der Klassen

$$L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE \subseteq EXPTIME \dots$$

- Bekannte echte Inklusionen: $L \subset PSPACE$, $P \subset EXPTIME$.
- Viele Beziehungen (z.B. P vs NP) sind noch offen.

Komplementäre Klassen (co-C)

- Problem $\bar{P}$ ist das Komplement (Antwort Ja/Nein vertauscht).
- $co-C = \{P \mid \bar{P} \in C\}$.
- Deterministische Klassen sind abgeschlossen: $P = co-P$.
- Bei nicht-deterministischen Klassen (NP) ist $NP \stackrel{?}{=} co-NP$ offen.

Korrektheit von Programmen

Einführung & Syntax von SIMPLE($\mathbb{Z}$)

Die Sprache **SIMPLE($\mathbb{Z}$)** ist eine imperative Programmiersprache, die auf ganzen Zahlen ($\mathbb{Z}$) operiert.

Syntax

Die Syntax wird durch kontextfreie Produktionen definiert.

Programmelemente

- **Var:** Variablen ($x, y, z, \dots$)
- **Const:** Konstanten (0, 1, 42, $\dots$)
- **Term:** Arithmetische und logische Ausdrücke
- **Programm:** Die Struktur des Codes

Grammatik der Sprache:

SIMPLE($\mathbb{Z}$): SIMPLE über den ganzen Zahlen – Syntax

Spezifikationsmethode: kontextfreie Produktionen

Programme

$Programm \rightarrow$	„skip“	Leerprogramm
	„abort“	Abbruch, Exception
	$Var \text{ „:=“ } Term$	Zuweisung
	$Programm \text{ „;“ } Programm$	Hintereinanderausführung
	„if“ $Term$ „then“ $Programm$ „else“ $Programm$ „fi“	Konditional
	„while“ $Term$ „do“ $Programm$ „od“	Schleife

Variablen

$Var \rightarrow$ „x“ | „y“ | „z“ | „x₁“ | $\dots$ | „alles_was_gut_und_teuer_ist“ | $\dots$

$\mathcal{L}(Programm) \dots$ Menge aller Programme

$\mathcal{L}(Var) = \{x, y, z, x_1, \dots\} \dots$ Menge aller Variablen

Terme

$$\text{Term} \rightarrow \text{Var} \mid \text{Const} \mid \text{UnOp Term} \mid \text{"(" Term BinOp Term "}"$$

$$\text{UnOp} \rightarrow \text{"+"} \mid \text{"-"} \mid \text{"\neg"}$$

$$\text{BinOp} \rightarrow \text{"+"} \mid \text{"-"} \mid \text{"*"} \mid \text{"/"} \mid \text{"^"} \mid \text{"\vee"} \mid \text{"\Rightarrow"} \mid \text{"="} \mid \text{"\neq"} \mid \text{">"} \mid \text{"\geq"} \mid \text{"<"} \mid \text{"\leq"}$$

$$\text{Const} \rightarrow \text{"0"} \mid \text{"1"} \mid \text{"2"} \mid \dots \mid \text{"42"} \mid \dots$$

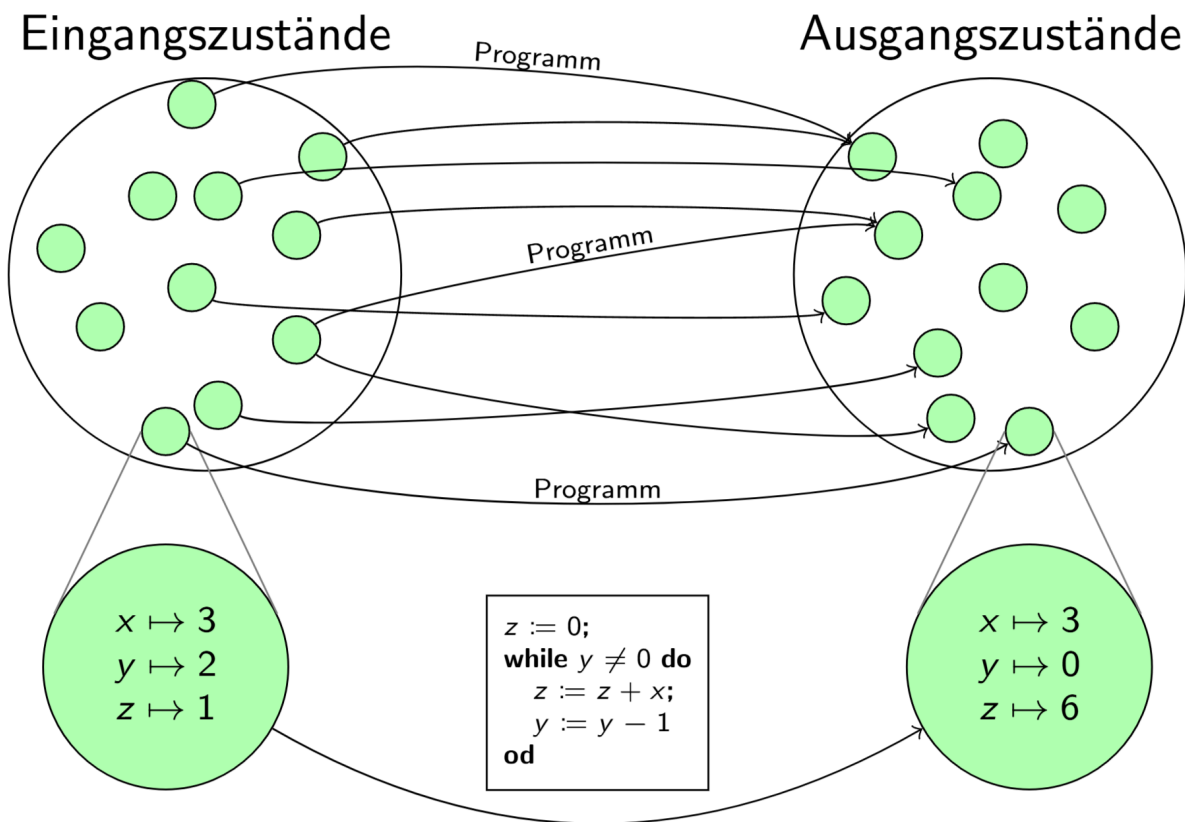
Konventionen:

- Kommentare beginnen mit `//`.
- Whitespace (Leerzeichen, Zeilenumbrüche, Kommentare) beliebig.
- Manche Klammern können fehlen: Außenklammern, assoziative Operatoren, Punkt- vor Strichrechnung
- Zu Beginn klammern wir mehrfache Hintereinanderausführung, um anzuzeigen, wie das Programm strukturiert ist.
Vergleiche `(z := 0; skip); abort` und `z := 0; (skip; abort)`

4

Programme als Zustandstransformatoren

Ein Programm transformiert einen **Eingangszustand** in einen **Ausgangszustand** (oder bricht ab/terminiert nicht).



- **Programmzustand (σ):** Abbildung von Variablen auf Werte.
 - $\mathcal{S} \triangleq \{\sigma \mid \sigma : \mathcal{L}(\text{Var}) \rightarrow \mathbb{Z}\}$
- **Konfiguration:** Ein Paar (Π, σ) (Restprogramm + Zustand) oder nur ein Zustand σ (finale Konfiguration).
 - $\mathcal{C} \triangleq (\mathcal{L}(\text{Programm}) \times \mathcal{S}) \cup \mathcal{S}$

Operationale Semantik

Die Semantik beschreibt die Bedeutung des Programms durch Zustandsübergänge.

1. Strukturelle Operationale Semantik (SOS / Small Step)

Beschreibt einzelne Berechnungsschritte mittels einer Übergangsrelation $\Rightarrow$.

Übergangsregeln:

- **Skip:** $(skip, \sigma) \Rightarrow \sigma$ (Zustand bleibt unverändert)
- **Zuweisung:** $(v := e, \sigma) \Rightarrow \sigma'$
 - Wobei $\sigma'(w) = [e]\sigma$ falls $w = v$, sonst $\sigma(w)$.
- **Sequenz:**
 - Wenn $(\Pi, \sigma) \Rightarrow \sigma'$, dann $(\Pi; \Omega, \sigma) \Rightarrow (\Omega, \sigma')$
 - Wenn $(\Pi, \sigma) \Rightarrow (\Pi', \sigma')$, dann $(\Pi; \Omega, \sigma) \Rightarrow (\Pi'; \Omega, \sigma')$
- **If-Else:**
 - Wenn $[e]\sigma \neq 0$: $(if \dots, \sigma) \Rightarrow (\Pi, \sigma)$
 - Wenn $[e]\sigma = 0$: $(if \dots, \sigma) \Rightarrow (\Omega, \sigma)$
- **While:**
 - Wenn $[e]\sigma \neq 0$: $(while \dots, \sigma) \Rightarrow (\Pi; while \dots, \sigma)$
 - Wenn $[e]\sigma = 0$: $(while \dots, \sigma) \Rightarrow \sigma$

Semantik von Termen $[e]\sigma$

Wir überladen die Klammern $[.]$:

- $[\Pi]\sigma$: Auswertung von Programmen
- $[e]\sigma$: Auswertung von Termen (liefert Wert in $\mathbb{Z}$)
- $[b]$: Funktion zum Operator (z.B. Boolesche Werte: 0 = falsch, alles andere (z.B. 1) = wahr).

2. Natürliche Semantik (Big Step)

Fokussiert auf das Endergebnis, nicht die Schritte. Definiert als Funktion $[\Pi]\sigma$.

Theorem (Definition der natürlichen Semantik):

Theorem

Die SOS von $\text{SIMPLE}(\mathbb{Z})$ besitzt folgende Eigenschaften.

- $[\text{skip}] \sigma = \sigma$
- $[v := e] \sigma = \sigma'$, wobei $\sigma'(w) = \begin{cases} [e] \sigma & \text{wenn } w = v \\ \sigma(w) & \text{wenn } w \neq v \end{cases}$
- $[\Pi; \Omega] \sigma = [\Omega] [\Pi] \sigma$
- $[\text{if } e \text{ then } \Pi \text{ else } \Omega \text{ fi}] \sigma = \begin{cases} [\Pi] \sigma & \text{wenn } [e] \sigma \neq 0 \\ [\Omega] \sigma & \text{wenn } [e] \sigma = 0 \end{cases}$
- $[\text{while } e \text{ do } \Pi \text{ od}] \sigma = \begin{cases} [\text{while } e \text{ do } \Pi \text{ od}] [\Pi] \sigma & \text{wenn } [e] \sigma \neq 0 \\ \sigma & \text{wenn } [e] \sigma = 0 \end{cases}$

$\Pi, \Omega \dots$ Programme, $v \dots$ Variable, $e \dots$ Term, $\sigma \dots$ Zustand

Korrektheitsaussagen (Hoare-Logik)

Wir spezifizieren Programmverhalten mit **Vorbedingungen** (Precondition) und **Nachbedingungen** (Postcondition).

Notation: $\{F\} \Pi \{G\}$

- F : Vorbedingung (Zustände vor Ausführung)
- Π : Programm
- G : Nachbedingung (Zustände nach Ausführung)

Prädikatenlogische Formeln als Zustandsmengen

Eine Formel F repräsentiert die Menge aller Zustände, in denen F wahr ist.

- $\{F\} \cap \{G\}$ entspricht logischem UND.
- $\{F\} \cup \{G\}$ entspricht logischem ODER.
- $\{1\} = \mathcal{S}$ (wahr, alle Zustände).
- $\{0\} = \emptyset$ (falsch, kein Zustand).

Partielle vs. Totale Korrektheit

Typ	Notation	Bedeutung
Partielle Korrektheit	$\{F\} \vdash \{G\}$ oder $\{F\} \Pi \{G\}$	Wenn der Startzustand F erfüllt und das Programm terminiert, dann gilt im Endzustand G . (Terminierung wird <i>nicht</i> garantiert).
Totale Korrektheit	$\{F\} \sqsubseteq \{G\}$	Wenn der Startzustand F erfüllt, dann terminiert das Programm und im Endzustand gilt G .

Zusammenhang

Totale Korrektheit = Partielle Korrektheit + Terminierung.

Logische Beziehungen & Semantik-Konzepte

Stärke von Formeln

- **Stärker:** F ist stärker als G , wenn $F \Rightarrow G$ (F impliziert G). F schränkt mehr ein (weniger Zustände).
- **Schwächer:** F ist schwächer als G , wenn $G \Rightarrow F$. F erlaubt mehr Zustände.
- **Beispiel:** $x = y$ ist stärker als $x \geq y$.

Prädikamentransformatoren

Wir können die Semantik von SIMPLE auch axiomatisch über diese Transformatoren definieren.

Weakest Precondition (wp) - für Totale Korrektheit

$wp(\Pi, G)$: Die schwächste Formel, sodass $\{wp(\Pi, G)\} \Pi \{G\}$ **total** korrekt ist.

- Menge aller Zustände, die terminieren und danach G erfüllen.

Weakest Liberal Precondition (wlp) - für Partielle Korrektheit

$wlp(\Pi, G)$: Die schwächste Formel, sodass $\{wlp(\Pi, G)\} \Pi \{G\}$ **partiell** korrekt ist.

- Menge aller Zustände, die bei Terminierung G erfüllen (oder nicht terminieren).

Strongest Postcondition (sp)

$sp(F, \Pi)$: Die stärkste Formel, die nach Ausführung von Π gilt, wenn vorher F galt.

Hoare-Kalkül: Regeln & Axiome

Ein Beweissystem, um Korrektheitsaussagen in prädikatenlogische Formeln zu zerlegen.

Grundlegende Regeln

1. Leerprogramm (Skip)

- $\{F\} skip \{F\}$ (Axiom)
- $wp(skip, G) = G$
- $sp(F, skip) = F$

2. Implikationsregel (Consequence Rule)

Erlaubt das "Stärken" der Vorbedingung und "Schwächen" der Nachbedingung.

Schreibweise für Regeln:

$$\frac{F_1 \Rightarrow G_1 \quad \{F_2\} \Pi_2 \{G_2\} \quad F_3 \Rightarrow G_3 \quad \{F_4\} \Pi_4 \{G_4\} \quad \dots}{\{F\} \Pi \{G\}} \quad \begin{array}{l} \text{Prämissen} \\ \text{Konklusion} \end{array}$$

Korrektheit: Immer, wenn alle Prämissen gültig (im Fall von Formeln) bzw. wahr (im Fall von Korrektheitsaussagen) sind, muss die Konklusion wahr sein.

Implikationsregel

$$\frac{F \Rightarrow F' \quad \{F'\} \Pi \{G'\} \quad G' \Rightarrow G}{\{F\} \Pi \{G\}} \quad (\text{imp}) \quad \begin{array}{l} \text{Wenn } F \Rightarrow F' \text{ gültig, } \{F'\} \Pi \{G'\} \text{ wahr und } G' \Rightarrow G \text{ gültig ist,} \\ \text{dann ist } \{F\} \Pi \{G\} \text{ eine wahre Korrektheitsaussage.} \end{array}$$

$$\frac{x > 5 \Rightarrow x > 0 \quad \{x > 0\} x := 2 \{x = 2\} \quad x = 2 \Rightarrow x \leq 3}{\{x > 5\} x := 2 \{x \leq 3\}} \quad (\text{imp})$$

3. Hintereinanderausführung (Sequenz)

Zerlegung in Teilschritte.

- Regel: Wenn $\{F\} \Pi \{G\}$ und $\{G\} \Omega \{H\}$, dann $\{F\} \Pi; \Omega \{H\}$.
- $wp(\Pi; \Omega, G) = wp(\Pi, wp(\Omega, G))$
- $sp(F, \Pi; \Omega) = sp(sp(F, \Pi), \Omega)$

Zuweisungen ($v := e$)

Hier gibt es zwei Denkweisen:

A. Rückwärts-Argumentation (Bevorzugt für manuelle Beweise)

Man setzt den Ausdruck e für v in die Nachbedingung ein (Substitution).

- **Axiom:** $\{G[e/v]\} v := e \{G\}$
- $wp(v := e, G) = G[e/v]$
- **Beispiel:** Um $\{?\} x := x + 5 \{x = 7\}$ zu zeigen:
 - Setze $x + 5$ für x ein: $x + 5 = 7 \Rightarrow x = 2$.
 - Vorbedingung: $\{x = 2\}$.

B. Vorwärts-Argumentation

Berechnet den neuen Zustand aus dem alten.

- $sp(F, v := e) = \exists v' (F[v'/v] \wedge v = e[v'/v])$
- v' ist der "alte" Wert der Variable.

Abort (Fehler/Abbruch)

- **Partielle Korrektheit:** $\{F\} \text{abort} \{G\}$ ist immer wahr (Axiom $\{1\} \text{abort} \{0\}$). Da es nie terminiert, ist die Implikation "wenn terminiert, dann G" trivial wahr.
- **Totale Korrektheit:** $\{0\} \text{abort} \{G\}$. Nur wahr, wenn Vorbedingung falsch ist (Zustandsmenge leer), da abort nie korrekt terminiert.

Konditionale (If-Then-Else)

Hoare-Regel:

Hoare-Kalkül

$$\frac{\{F \wedge e\} \Pi \{G\} \quad \{F \wedge \neg e\} \Omega \{G\}}{\{F\} \text{ if } e \text{ then } \Pi \text{ else } \Omega \text{ fi } \{G\}} \text{ (if)}$$

Alternative Regel:

$$\frac{\{F\} \Pi \{H\} \quad \{G\} \Omega \{H\}}{\{(e \Rightarrow F) \wedge (\neg e \Rightarrow G)\} \text{ if } e \text{ then } \Pi \text{ else } \Omega \text{ fi } \{H\}} \text{ (if')}$$

Es genügt eine der beiden Regeln für einen vollständigen Kalkül.

$(e \Rightarrow F) \wedge (\neg e \Rightarrow G)$ ist logisch äquivalent zu
 $(e \wedge F) \vee (\neg e \wedge G)$

Man muss beide Zweige beweisen:

1. $\{F \wedge e\} \Pi \{G\}$ (Then-Zweig)
2. $\{F \wedge \neg e\} \Omega \{G\}$ (Else-Zweig)

WP/WLP:

- $wp(\text{if } \dots, G) = (e \Rightarrow wp(\Pi, G)) \wedge (\neg e \Rightarrow wp(\Omega, G))$

Annotierungskalkül

Eine praktische Methode, um Beweise direkt im Programmcode zu führen (statt riesige Beweisbäume zu zeichnen).

- Man schreibt Zusicherungen (Assertions) in geschweifte Klammern $\{\dots\}$ zwischen die Anweisungen.
- **Pfeile:** $\uparrow$ (Rückwärtsanwendung von Regeln) oder $\downarrow$ (Vorwärtsanwendung).

Beispiel Wertetausch:

Wertetausch (Beweis mit Annotierungen)

$$\begin{aligned}
 &\{ P: x = x_0 \wedge y = y_0 \} \\
 &\{ F_3: Q \begin{bmatrix} t \\ y \end{bmatrix} \begin{bmatrix} y \\ x \end{bmatrix} \begin{bmatrix} x \\ t \end{bmatrix} \} \quad \text{zw } \uparrow \\
 &t := x; \\
 &\{ F_2: Q \begin{bmatrix} t \\ y \end{bmatrix} \begin{bmatrix} y \\ x \end{bmatrix} \} \quad \text{zw } \uparrow \\
 &x := y; \\
 &\{ F_1: Q \begin{bmatrix} t \\ y \end{bmatrix} \} \quad \text{zw } \uparrow \\
 &y := t \\
 &\{ Q: x = y_0 \wedge y = x_0 \}
 \end{aligned}$$

Implikationsregel: Beweise die Implikation

$$P \Rightarrow F_3: \quad (x = x_0 \wedge y = y_0) \Rightarrow Q \begin{bmatrix} t \\ y \end{bmatrix} \begin{bmatrix} y \\ x \end{bmatrix} \begin{bmatrix} x \\ t \end{bmatrix}$$

Schleifen (While)

Das schwierigste Konstrukt, da die Anzahl der Durchläufe zur Compile-Zeit unbekannt ist. $wp(loop, G)$ ist im Allgemeinen eine unendliche Formel ($\exists i \dots$). Daher nutzen wir **Invarianten**.

Partielle Korrektheit: Invarianten

Wir suchen eine Formel Inv (**Schleifeninvariante**), die:

1. Vor der Schleife gilt.
2. Nach jedem Schleifendurchlauf wieder gilt (sofern die Schleifenbedingung e wahr war).
3. Wenn die Schleife endet (Bedingung $\neg e$), zusammen mit $\neg e$ die Nachbedingung G impliziert.

Hoare-Regel (wh):

$$\frac{\{Inv \wedge e\} \Pi \{Inv\}}{\{Inv\} \text{ while } e \text{ do } \Pi \text{ od } \{Inv \wedge \neg e\}}$$

Vorgehen beim Beweis (Partiell)

1. Finde geeignete Invariante Inv .
2. Zeige: $Vorbedingung \Rightarrow Inv$ (Initialisierung).
3. Zeige: $\{Inv \wedge e\} \Pi \{Inv\}$ (Erhaltung).
4. Zeige: $(Inv \wedge \neg e) \Rightarrow Nachbedingung$ (Abschluss).

Totale Korrektheit: Varianten (Terminierung)

Zusätzlich zur Invariante benötigen wir einen **Term (Variante)** t , der die "Größe" des Restproblems beschreibt.

Anforderungen an t :

1. Muss eine ganze Zahl sein ($t \in \mathbb{Z}$).

2. Muss nach unten beschränkt sein (meist $t \geq 0$).
3. Muss in jedem Schleifendurchlauf **strikt kleiner** werden ($t_{neu} < t_{alt}$).

Hoare-Regel (Totale Korrektheit):

Totale Korrektheit von Schleifen

t : „Terminationsfunktion“ oder „Variante“, gibt die Größe des momentanen Zustands an.

Der Wert des Ausdrucks t muss

- ... eine ganze Zahl sein,
- ... nicht-negativ vor und während der Schleife sein und
- ... in jedem Schleifendurchlauf strikt kleiner werden.

$$\frac{\begin{array}{lll} \text{partielle Korrektheit} & \text{Variante wird strikt kleiner} & \text{und ist beschränkt} \\ \{Inv \wedge e\} \sqcap \{Inv\} & \{Inv \wedge e \wedge t = t_0\} \sqcap \{t < t_0\} & Inv \Rightarrow t \geq 0 \end{array}}{\{Inv\} \mathbf{while} \ e \ \mathbf{do} \ \sqcap \ \mathbf{od} \ \{Inv \wedge \neg e\}}$$

t_0 : Hilfsvariable, repräsentiert den Wert von t vor der Iteration

Es muss gelten:

$$Inv \wedge e \wedge t = t_0 \implies wp(\Pi, t < t_0)$$

(Intuitiv: Wenn die Schleife betreten wird, ist t danach kleiner als der gemerkte Wert t_0 , aber immer noch gültig).